```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.ArrowOut.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S28_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.ArrowOut.Boot
{
    /// &lt;summary&gt;
```

```
/// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
/// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
    static bool ConfigStep()
    {
```

```
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.ArrowOut.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
    }
    public sealed class MockTimeService : ITimeService
    {
```

```
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action&lt;AdLifecycle&gt; OnAdEvent;
    public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
        =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
    public void Preload(string pointId)
    {
        Logger.Info("ADS", "preload (mock): " + pointId);
```

```
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
// AdLedger.cs
// AdLedger.cs - S1 IAA wiring face for ArrowOut (task book T008, M07 / S06):
// the three optional rewarded-video grant positions (hint / rescue / skip) sit
// behind one daily grant ledger. Frozen values (E4 M07 anchor + U149 hard cap +
// cfg_compliance d7 block validated by ConfigValidator): daily_grant_cap = 12,
// wall_clock_seconds = 86400 (24h rolling window), ads are NEVER required for
// closure (GrayboxSim.AdsRequiredForClosure = false - a zero-ad campaign closes
// fine). Cap reject is silent and mutates nothing (zero system blocking law:
// no counter moves, no log line, no board change). D7 retention-guardrail pivot
// clause (guardrail #1): if D7 retention ever lands below the category baseline
// (baseline anchor = research pending), the monetization rhythm gets re-reviewed
// - never patch retention by adding more ad pressure.
// Pure C# (kit-port style, zero UnityEngine), C#5-clean for the csc dry-probe.
using System;
using System.Collections.Generic;
namespace Sy.ArrowOut
{
    public sealed class AdLedger
    {
        public const int DailyGrantCap = 12;            // U149 hard cap (12/day)
        public const double WallClockSeconds = 86400.0; // 24h rolling window
        readonly GrayboxSim sim;
        double windowStart = -1.0;
        int grantsToday;
        readonly List&lt;double&gt; grantStamps = new List&lt;double&gt;();   // T010 c1 save.adcaps face (B1833)
        public AdLedger(GrayboxSim owner) { sim = owner; }
        public int GrantsUsed { get { return grantsToday; } }
        public bool CapReached { get { return grantsToday &gt;= DailyGrantCap; } }
        void RollWindow(double now)
        {
            if (windowStart &lt; 0.0) { windowStart = now; return; }
            if (now - windowStart &gt;= WallClockSeconds)
            {
                windowStart = now;
                grantsToday = 0;
            }
        }
        bool TryConsume()
        {
            RollWindow(sim.SimTime);
            if (grantsToday &gt;= DailyGrantCap) return false;   // silent cap reject
            grantsToday++;
            grantStamps.Add(sim.SimTime);   // T010 c1: stamp the consumed grant (save.adcaps face)
            return true;
        }
        // ---- T010 c1 additive persistence face (B1833): ExportStamps IS the
```

```
        // save.adcaps data face (the T008 contract); LoadStamps restores the
        // rolling window from a persisted face so the 12/day cap survives a
        // restart. Stamps outside the newest 24h window drop on load (the window
        // face recomputes from the kept stamps). ----
        public double[] ExportStamps()
        {
            return grantStamps.ToArray();
        }
        public void LoadStamps(double[] restored)
        {
            grantStamps.Clear();
            if (restored == null || restored.Length == 0)
            {
                windowStart = -1.0;
                grantsToday = 0;
                return;
            }
            double newest = restored[0];
            for (int i = 1; i &lt; restored.Length; i++) if (restored[i] &gt; newest) newest = restored[i];
            double oldest = double.MaxValue;
            for (int i = 0; i &lt; restored.Length; i++)
            {
                if (newest - restored[i] &gt;= WallClockSeconds) continue;   // outside the 24h window
                grantStamps.Add(restored[i]);
                if (restored[i] &lt; oldest) oldest = restored[i];
            }
            windowStart = grantStamps.Count &gt; 0 ? oldest : -1.0;
            grantsToday = grantStamps.Count;
        }
        // The three grant positions: player-initiated, rewarded, purely optional.
        // A rejected grant returns a null/false result and touches nothing.
        public string GatedGrantHint()
        {
            if (!TryConsume()) return null;
            return sim.GrantHint();
        }
        public bool GatedGrantRescue(int pileId)
        {
            if (!TryConsume()) return false;
            sim.GrantRescue(pileId);
            return true;
        }
        public bool GatedGrantSkip()
        {
            if (!TryConsume()) return false;
            sim.GrantSkip();
            return true;
        }
    }
}
```

```
// AudioHooks.cs
// AudioHooks.cs - S1 T010 c2 audio hook true-path core for ArrowOut (B1834, G26
// ExileFarm AudioHooks B1284 pattern ported onto the G28 frozen faces).
// The nine TJ-custom audio orders (demand-doc section 5, manifest AU-01..AU-09)
// are cloud assets that have not landed yet - this core IS the frozen contract
// they plug into: the frozen stem census (11 manifest ids verbatim) plus the
// event-to-stem map that turns the REAL kit event stream into audio hook
// records. True path = the hook consumer walks the same event list the checks
// and the gate drive walk - zero mocked sources, zero UnityEngine (no AudioClip
// face is played here by design: the platform layer will later subscribe to
// these hook records and play the stem assets by name when the waves land).
// Mapping law (demand-doc section 5, zero re-tuning): the board load starts the
// mainboard BGM (AU-01); the 1.00 three-same-beat member sfx_ignite rings the
// ignite pop (AU-02) and slide_start rings the slide glide (AU-03 - the
// identity window members); the slam-stop rings the stop thump (AU-04, the
// board-rewritten audio signature) and the pile slam ALSO rings the chain
// trigger (AU-08 - the stopped arrow becomes a pile that rewrites every
// dependent lane: one arrow moving the whole board, the screen-record climax
// face); every escape rings the exit chime (AU-05); the board clear rings the
// full three-tone jingle (AU-06); the one-cell-short near miss rings the warm
// unresolved sting (AU-07, zero-punishment tone); the board-clear share card
// rings the share chime (AU-11). The UI tap (AU-10) and the dex unlock (AU-09)
// are explicit calls the button layer and the codex layer will wire (the sim
// stream carries no button or dex event). Everything else stays silent by law:
// caption ticks, player_tap rows (the ignite stem carries the game-tap
// feedback), ghost_pass / turn_fire / double_stop additive markers (the stop
// thump already rang), stuck rows, rescue offers and declines (U149 - the loop
// never requires audio ads), honest fail rows (zero-punishment wall) and every
// reject (zero dead-tap face).
using System;
using System.Collections.Generic;
namespace Sy.ArrowOut
{
    public sealed class AudioHook
    {
        public double t;
        public string stem;
        public string source;
        public AudioHook(double t, string stem, string source) { this.t = t; this.stem = stem; this.source = source; }
    }
    public sealed class AudioHooks
    {
        // ---- frozen stem census (demand-doc section 5, manifest ids verbatim) ----
        public const int StemCount = 11;
        public static readonly string[] Stems =
        {
            "g28_bgm_mainboard", "g28_arrow_ignite", "g28_arrow_slide", "g28_arrow_stop",
            "g28_arrow_out", "g28_board_clear", "g28_near_miss", "g28_chain_trigger",
            "g28_dex_unlock", "g28_ui_tap", "g28_share_chime"
        };
```

```
public readonly List&lt;AudioHook&gt; Fired = new List&lt;AudioHook&gt;();
public AudioHooks()
{
    // frozen census guard: the 11 stems must stay unique and non-empty
    for (int i = 0; i &lt; Stems.Length; i++)
    {
        if (Stems[i] == null || Stems[i].Length == 0)
            throw new ArgumentException("audio stem " + i + " must carry the manifest id");
        for (int j = i + 1; j &lt; Stems.Length; j++)
            if (Stems[i] == Stems[j])
                throw new ArgumentException("audio stem census must stay unique: " + Stems[i]);
    }
}
// ---- the frozen event-to-stem map (one stem per real event type; an
// unmapped type is silent by law - see the file header) -------------------
static readonly string[] MapTypes =
{
    "level_load", "sfx_ignite", "slide_start", "slide_stop", "pile_stop",
    "arrow_out", "level_complete", "near_miss", "share_card"
};
static readonly string[] MapStems =
{
    "g28_bgm_mainboard", "g28_arrow_ignite", "g28_arrow_slide", "g28_arrow_stop",
    "g28_chain_trigger", "g28_arrow_out", "g28_board_clear", "g28_near_miss",
    "g28_share_chime"
};
static string StemOf(string type)
{
    for (int i = 0; i &lt; MapTypes.Length; i++) if (MapTypes[i] == type) return MapStems[i];
    return null;   // unmapped = silent by law
}
/// &lt;summary&gt;Consume the REAL kit event stream (same list, same order, same
/// ticks the checks and the gate drive walk). The BGM starts when the board
/// lands; the ignite / slide / stop / chain / escape / clear / near-miss /
/// share faces ring on their own rows; every reject, offer, caption and
/// marker row stays silent.&lt;/summary&gt;
public void ConsumeSim(GrayboxSim sim)
{
    if (sim == null) return;
    for (int i = 0; i &lt; sim.Events.Count; i++)
    {
        string stem = StemOf(sim.Events[i].type);
        if (stem != null)
            Fired.Add(new AudioHook(sim.Events[i].t, stem, "sim:" + sim.Events[i].type));
    }
}
/// &lt;summary&gt;UI tap face: the page button layer calls this on every button
/// tap (wall-clock domain, t = -1 - it is not a sim beat). Frozen stem only.&lt;/summary&gt;
public void UiTap()
{
```

```
            Fired.Add(new AudioHook(-1.0, "g28_ui_tap", "ui:tap"));
        }
        /// &lt;summary&gt;Dex unlock face: the codex layer calls this when a NEW entry
        /// lands in the player's dex (a board type or arrow type first seen -
        /// wall-clock domain, t = -1). The S1 sim stream carries no dex event by
        /// design; the S2 codex face wires this call.&lt;/summary&gt;
        public void DexUnlock()
        {
            Fired.Add(new AudioHook(-1.0, "g28_dex_unlock", "dex:unlock"));
        }
        /// &lt;summary&gt;S2 T013 codex stream face (G26 ConsumeCodex twin): every
        /// NEW dex entry in the CodexCore stream rings the unlock chime - a
        /// fresh full campaign fires thirty-seven, a re-drive fires zero
        /// (idempotence is the audio face of the zero-punishment wall).&lt;/summary&gt;
        public void ConsumeCodex(CodexCore core)
        {
            if (core == null) return;
            for (int i = 0; i &lt; core.Events.Count; i++)
            {
                if (core.Events[i].type != "dex_unlock") continue;
                Fired.Add(new AudioHook(core.Events[i].t, "g28_dex_unlock", "codex:dex_unlock"));
            }
        }
        public int CountStem(string stem)
        {
            int n = 0;
            for (int i = 0; i &lt; Fired.Count; i++) if (Fired[i].stem == stem) n++;
            return n;
        }
        public bool HookExistsAt(string stem, double t)
        {
            for (int i = 0; i &lt; Fired.Count; i++)
                if (Fired[i].stem == stem &amp;&amp; Math.Abs(Fired[i].t - t) &lt; 1e-9) return true;
            return false;
        }
    }
}
// CodexCore.cs
// CodexCore.cs - S2 T013 board-type / arrow-type / achievement codex core for
// ArrowOut (B1869). The zero-punishment collection face (demand-doc section 6
// codex table): board family 30 (the E4 five board types + the S2 expansion
// roster), arrow family 4 (classic / ghost / turner / double), achievement
// family 3 (first_clear / all_types / no_rescue - pure progress, zero lottery).
// Row index space: boards 0..29, arrows 30..33, achievements 34..36, roster
// order = cfg_codex.json row order (the CodexKeep family twin law).
// Unlock paths are structural: a board row joins when its board is first
// cleared (the presenter face OnBoardCleared, wired by GrayboxSim.CheckAllOff),
// an arrow row joins on the first LEGAL ignite of that type (OnArrowIgnited),
// achievements evaluate on every add: first_clear on the first board clear,
// no_rescue on the first grant-free clear, all_types when every type row is
```

```
// owned. The first join of a row rings exactly one "dex_unlock" event (the
// AU-09 chime path, AudioHooks.ConsumeCodex); re-joins stay silent. There is
// no wipe call anywhere - the codex keeps everything on loss (zero punishment
// is structural, the same law as GrayboxSim.LevelsSeen).
// No UnityEngine dependency (pure C#, editor-check driven, G26 ExileFarm
// CodexCore pattern ported onto the G28 frozen sim - B1274 kit-port family).
using System;
using System.Collections.Generic;
namespace Sy.ArrowOut
{
    public sealed class CodexEvent
    {
        public double t;
        public string type;
        public string detail;
        public CodexEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail; }
    }
    public sealed class CodexCore
    {
        public const int BoardFamily = 30;   // L1-L30 board types, one row per level
        public const int ArrowFamily = 4;     // classic / ghost / turner / double
        public const int AchFamily = 3;      // first_clear / all_types / no_rescue
        public const int TotalRows = BoardFamily + ArrowFamily + AchFamily;   // 37
        public const int ArrowFirst = BoardFamily;            // 30
        public const int AchFirst = BoardFamily + ArrowFamily; // 34
        public const int AchFirstClear = AchFirst;            // 34
        public const int AchAllTypes = AchFirst + 1;          // 35
        public const int AchNoRescue = AchFirst + 2;          // 36
        public double SimTime;
        public readonly List&lt;CodexEvent&gt; Events = new List&lt;CodexEvent&gt;();
        public readonly HashSet&lt;int&gt; Owned = new HashSet&lt;int&gt;();   // never wiped on loss
        private string[] _names = new string[0];
        public void Log(string type, string detail) { Events.Add(new CodexEvent(SimTime, type, detail)); }
        public int CountEvents(string type)
        {
            int n = 0;
            for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
            return n;
        }
        public void InitCodex(string[] names)
        {
            if (names == null || names.Length != TotalRows)
                throw new ArgumentException("codex roster must carry exactly " + TotalRows
                    + " rows (board 30 / arrow 4 / achievement 3), got "
                    + (names == null ? 0 : names.Length));
            for (int i = 0; i &lt; names.Length; i++)
                if (names[i] == null || names[i].Length == 0)
                    throw new ArgumentException("codex row " + i + " carries an empty name");
            _names = names;
        }
```

```
public bool HasOwned(int rowIdx) { return Owned.Contains(rowIdx); }
public int OwnedCount { get { return Owned.Count; } }
public int[] QueryAll()
{
    List&lt;int&gt; list = new List&lt;int&gt;(Owned);
    list.Sort();
    return list.ToArray();
}
// First join rings one dex_unlock (AU-09); re-joins stay silent - the
// idempotence is the audio face of the zero-punishment wall.
public bool AddOwned(int rowIdx)
{
    if (rowIdx &lt; 0 || rowIdx &gt;= TotalRows) return false;
    if (!Owned.Add(rowIdx)) return false;
    Log("dex_unlock", "codex entry " + (_names.Length == TotalRows ? _names[rowIdx] : ("row " + rowIdx))
        + " joins the collection");
    return true;
}
// Presenter face: GrayboxSim.Ignite calls this on every LEGAL ignite.
public bool OnArrowIgnited(ArrowType t)
{
    int rowIdx = ArrowFirst + (int)t;
    bool rang = AddOwned(rowIdx);
    EvaluateAllTypes();
    return rang;
}
// Presenter face: GrayboxSim.CheckAllOff calls this on every board
// clear. grantsTainted = an optional grant (hint / rescue / skip) was
// used on this board - the no_rescue star only rings on grant-free
// clears, but the board row itself never cares (a rescued clear still
// joins the collection - zero punishment).
public bool OnBoardCleared(string boardName, bool grantsTainted)
{
    for (int i = 0; i &lt; BoardFamily; i++)
    {
        if (_names.Length != TotalRows || _names[i] != boardName) continue;
        AddOwned(i);
        if (!HasOwned(AchFirstClear)) AddOwned(AchFirstClear);
        if (!grantsTainted &amp;&amp; !HasOwned(AchNoRescue)) AddOwned(AchNoRescue);
        EvaluateAllTypes();
        return true;
    }
    return false;
}
void EvaluateAllTypes()
{
    if (HasOwned(AchAllTypes)) return;
    for (int i = 0; i &lt; AchFirst; i++) if (!Owned.Contains(i)) return;
    AddOwned(AchAllTypes);
}
```

```
    }
}
// GrayboxBot.cs
// GrayboxBot.cs - solver and bare-play bot for the ArrowOut E4 graybox (M6-B lane).
// The bot is the automated player: it solves boards with its own brain (DFS over
// ignition orders), plays the campaign with zero hints / zero ads / zero rng, runs
// the chain demo, and walks the trap board into the honest near-miss fail. The
// solver works on copied arrow arrays through the single BoardOps.ComputeSlide
// implementation (no logic duplication between live sim and solver).
using System;
using System.Collections.Generic;
using System.Text;
namespace Sy.ArrowOut
{
    public static class GrayboxBot
    {
        static Arrow[] Clone(Arrow[] src)
        {
            Arrow[] c = new Arrow[src.Length];
            for (int i = 0; i &lt; src.Length; i++)
            {
                c[i] = new Arrow();
                c[i].Id = src[i].Id; c[i].X = src[i].X; c[i].Y = src[i].Y;
                c[i].Dir = src[i].Dir; c[i].Type = src[i].Type; c[i].OnBoard = src[i].OnBoard;
            }
            return c;
        }
        static Arrow[] Initial(LevelDef def)
        {
            Arrow[] a = new Arrow[def.ArrowCount];
            for (int i = 0; i &lt; def.ArrowCount; i++)
            {
                a[i] = new Arrow();
                a[i].Id = i; a[i].X = def.Ax[i]; a[i].Y = def.Ay[i];
                a[i].Dir = def.Adir[i]; a[i].Type = def.Atype[i]; a[i].OnBoard = true;
            }
            return a;
        }
        static void Apply(Arrow a, SlideResult r)
        {
            if (r.Exited) { a.OnBoard = false; a.X = -1; a.Y = -1; }
            else { a.X = r.X; a.Y = r.Y; a.Dir = r.Dir; }
        }
        static bool AllOff(Arrow[] st)
        {
            for (int i = 0; i &lt; st.Length; i++) if (st[i].OnBoard) return false;
            return true;
        }
        static string Key(Arrow[] st)
        {
```

```
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i &lt; st.Length; i++)
    {
        if (st[i].OnBoard) sb.Append('1').Append(st[i].X).Append(',').Append(st[i].Y).Append(',').Append(st[i].D
        else sb.Append('0').Append(';');
    }
    return sb.ToString();
}
// DFS with visited-state dedup; returns the first found ignition order that
// clears the board, or null when the board has no solution (trap face).
public static List&lt;int&gt; SolveFast(LevelDef def)
{
    List&lt;int&gt; path = new List&lt;int&gt;();
    HashSet&lt;string&gt; visited = new HashSet&lt;string&gt;();
    if (Dfs(def, Initial(def), visited, path, 0)) return path;
    return null;
}
public static List&lt;int&gt; SolveFromSnapshot(LevelDef def, Arrow[] live)
{
    List&lt;int&gt; path = new List&lt;int&gt;();
    HashSet&lt;string&gt; visited = new HashSet&lt;string&gt;();
    if (Dfs(def, Clone(live), visited, path, 0)) return path;
    return null;
}
public static string SolveFirstMove(LevelDef def, Arrow[] live)
{
    List&lt;int&gt; sol = SolveFromSnapshot(def, live);
    if (sol == null || sol.Count == 0) return null;
    return "ignite arrow " + sol[0];
}
static bool Dfs(LevelDef def, Arrow[] st, HashSet&lt;string&gt; visited, List&lt;int&gt; path, int depth)
{
    if (depth &gt; 64) return false;
    if (AllOff(st)) return true;
    string k = Key(st);
    if (visited.Contains(k)) return false;
    visited.Add(k);
    for (int id = 0; id &lt; st.Length; id++)
    {
        SlideResult r = BoardOps.ComputeSlide(def, st, id);
        if (!r.Legal) continue;
        Arrow[] ns = Clone(st);
        Apply(ns[id], r);
        path.Add(id);
        if (Dfs(def, ns, visited, path, depth + 1)) return true;
        path.RemoveAt(path.Count - 1);
    }
    return false;
}
// Full-sequence enumeration (no dedup, reject-pruned, capped): counts every
```

```
// distinct winning ignition order up to the length cap. Small boards only.
public static List&lt;List&lt;int&gt;&gt; EnumerateSolutions(LevelDef def, int cap)
{
    List&lt;List&lt;int&gt;&gt; found = new List&lt;List&lt;int&gt;&gt;();
    List&lt;int&gt; path = new List&lt;int&gt;();
    EnumDfs(def, Initial(def), path, cap, found);
    return found;
}
static void EnumDfs(LevelDef def, Arrow[] st, List&lt;int&gt; path, int cap, List&lt;List&lt;int&gt;&gt; found)
{
    if (found.Count &gt; 4096) return;
    if (AllOff(st)) { found.Add(new List&lt;int&gt;(path)); return; }
    if (path.Count &gt;= cap) return;
    for (int id = 0; id &lt; st.Length; id++)
    {
        SlideResult r = BoardOps.ComputeSlide(def, st, id);
        if (!r.Legal) continue;
        Arrow[] ns = Clone(st);
        Apply(ns[id], r);
        path.Add(id);
        EnumDfs(def, ns, path, cap, found);
        path.RemoveAt(path.Count - 1);
    }
}
// The bare-play campaign: five boards in a row. Solvable boards clear on the
// solver's own first order (zero hints, zero ads, zero rng); the trap board
// ends stuck -&gt; near-miss -&gt; offers declined -&gt; honest fail, dex keeps all.
public static void PlayCampaign(GrayboxSim sim)
{
    LevelDef[] levels =
    {
        ArrowLevels.Hook(), ArrowLevels.BuddyBlock(), ArrowLevels.OneCellShort(),
        ArrowLevels.TypeVocabulary(), ArrowLevels.ThirteenHighway()
    };
    for (int li = 0; li &lt; levels.Length; li++)
    {
        LevelDef def = levels[li];
        sim.SimTime += 0.5;
        sim.LoadLevel(def);
        List&lt;int&gt; sol = SolveFast(def);
        if (sol == null)
        {
            // trap board: scripted futile order (free bird out, victim one cell
            // short, blocker frozen) -&gt; stuck -&gt; near-miss -&gt; decline -&gt; fail
            sim.SimTime += 0.5; sim.Ignite(2);
            sim.SimTime += 0.5; sim.Ignite(0);
            sim.SimTime += 0.5; sim.Ignite(1);
            sim.CheckStuck();
            sim.DeclineRescue();
            sim.FailLevel();
```

```
                }
                else
                {
                    for (int i = 0; i &lt; sol.Count; i++)
                    {
                        sim.SimTime += 0.5;
                        sim.Ignite(sol[i]);
                    }
                }
            }
        }
        // Chain demo on the buddy-block board: both dependents pile-stop behind the
        // same arrow, it escapes, then both dependents follow it out (depth 2).
        public static void ChainDemo(GrayboxSim sim)
        {
            sim.SimTime += 0.5;
            sim.LoadLevel(ArrowLevels.BuddyBlock());
            int[] order = { 1, 2, 0, 1, 2 };
            for (int i = 0; i &lt; order.Length; i++)
            {
                sim.SimTime += 0.5;
                sim.Ignite(order[i]);
            }
        }
    }
}
// GrayboxSim.cs
// GrayboxSim.cs - pure C# deterministic logic prototype for ArrowOut (E4 graybox, M6-B lane).
// Arrow-escape board puzzle: tap an arrow, it slides on its own until it slams into a
// wall, the border, or a stopped buddy arrow (a "pile"). Arrows leave the board
// through edge gaps. Plan the ignition order so every arrow escapes. The self-slide
// is the 3-second perceivable identity vs draw/drag neighbors (E1 face): one tap,
// then the arrow crosses the whole lane with zero further input. A stopped arrow is
// a pile that blocks lanes and unblocks its dependents when it escapes (chain depth
// face). Ghost / turner / double arrows are the mechanism vocabulary (U023 axes).
// The "one cell short" mutual block is the near-miss beat with optional IAA outs
// (pile rescue / level skip / move hint) - purely optional, the campaign closes with
// zero ads. Banned vocab (E1 guardrails): the original Miniclip title and the G06
// sibling name never appear on any face. Share face: board-clear achievement card
// with an X032-style code, never a score form (B904 guardrail #2 analog).
// No UnityEngine dependency: sim time is logical seconds; wall-clock perf is
// measured by the editor check. Zero RNG by construction.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.ArrowOut
{
    public sealed class SimEvent
    {
```

```
    public double t;
    public string type;
    public string detail;
    public SimEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detail; }
}
public enum ArrowType { Base = 0, Ghost = 1, Turner = 2, Double = 3 }
// dir: 0=R 1=D 2=L 3=U (turner turns +1 mod 4 = clockwise: R-&gt;D-&gt;L-&gt;U)
public sealed class Arrow
{
    public int Id; public int X, Y, Dir;
    public ArrowType Type; public bool OnBoard;
}
public sealed class SlideResult
{
    public bool Legal, Exited, OutOfRange, AlreadyOut;
    public bool GhostPass, TurnFired, DoubleStop;
    public int X, Y, Dir, Moves;
    public int BlockedByPileId = -1;
}
public sealed class LevelDef
{
    public string Name; public int W, H;
    public bool[] Walls;                    // W*H, row-major
    public bool[] GapRight, GapLeft;       // length H, indexed by row y
    public bool[] GapTop, GapBottom;        // length W, indexed by col x
    public int ArrowCount;
    public int[] Ax, Ay, Adir;
    public ArrowType[] Atype;
}
public static class BoardOps
{
    public static readonly int[] DX = { 1, 0, -1, 0 };
    public static readonly int[] DY = { 0, 1, 0, -1 };
    public static string DirName(int dir)
    {
        return dir == 0 ? "right" : dir == 1 ? "down" : dir == 2 ? "left" : "up";
    }
    public static string TypeName(ArrowType t)
    {
        return t == ArrowType.Base ? "base" : t == ArrowType.Ghost ? "ghost"
            : t == ArrowType.Turner ? "turner" : "double";
    }
    // single source of truth for sliding: used by the live sim AND the solver copies.
    public static SlideResult ComputeSlide(LevelDef level, Arrow[] arrows, int id)
    {
        SlideResult r = new SlideResult();
        if (id &lt; 0 || id &gt;= arrows.Length) { r.OutOfRange = true; return r; }
        Arrow a = arrows[id];
        if (!a.OnBoard) { r.AlreadyOut = true; return r; }
        int x = a.X, y = a.Y, dir = a.Dir;
```

```
int moves = 0;
int passUsed = 0;
int turnsLeft = (a.Type == ArrowType.Turner) ? 1 : 0;
int maxMoves = (a.Type == ArrowType.Double) ? 2 : int.MaxValue;
while (true)
{
    if (a.Type == ArrowType.Double &amp;amp;&amp;amp; moves &gt;= maxMoves) { r.DoubleStop = true; break; }
    int nx = x + DX[dir], ny = y + DY[dir];
    bool off = (nx &lt; 0 || ny &lt; 0 || nx &gt;= level.W || ny &gt;= level.H);
    if (off)
    {
        bool gap;
        if (nx &gt;= level.W) gap = level.GapRight[y];
        else if (nx &lt; 0) gap = level.GapLeft[y];
        else if (ny &gt;= level.H) gap = level.GapBottom[x];
        else gap = level.GapTop[x];
        if (gap) { moves++; r.Exited = true; }
        break;
    }
    if (level.Walls[ny * level.W + nx])
    {
        if (a.Type == ArrowType.Ghost &amp;amp;&amp;amp; passUsed == 0)
        {
            passUsed++; moves++; r.GhostPass = true;
            x = nx; y = ny;
            continue;
        }
        if (turnsLeft &gt; 0) { turnsLeft--; dir = (dir + 1) % 4; r.TurnFired = true; continue; }
        break;
    }
    Arrow pile = null;
    for (int i = 0; i &lt; arrows.Length; i++)
    {
        if (i != id &amp;amp;&amp;amp; arrows[i].OnBoard &amp;amp;&amp;amp; arrows[i].X == nx &amp;amp;&amp;amp; arrows[i].Y == ny)
    }
    if (pile != null)
    {
        if (a.Type == ArrowType.Ghost &amp;amp;&amp;amp; passUsed == 0)
        {
            passUsed++; moves++; r.GhostPass = true;
            x = nx; y = ny;
            continue;
        }
        if (turnsLeft &gt; 0) { turnsLeft--; dir = (dir + 1) % 4; r.TurnFired = true; continue; }
        r.BlockedByPileId = pile.Id;
        break;
    }
    moves++; x = nx; y = ny;
}
if (moves == 0 &amp;amp;&amp;amp; !r.Exited) return r;   // zero-move ignition = illegal
```

```
            r.Legal = true;
            if (!r.Exited) { r.X = x; r.Y = y; r.Dir = dir; }
            r.Moves = moves;
            return r;
        }
        public static Arrow PileAt(LevelDef level, Arrow[] arrows, int selfId, int cx, int cy)
        {
            if (cx &lt; 0 || cy &lt; 0 || cx &gt;= level.W || cy &gt;= level.H) return null;
            for (int i = 0; i &lt; arrows.Length; i++)
                if (i != selfId &amp;&amp; arrows[i].OnBoard &amp;&amp; arrows[i].X == cx &amp;&amp; arrows[i].Y == cy)
            return null;
        }
        // exit distance ignoring other arrows (walls/edges/gaps only), scanning at most
        // 2 steps: returns 2 when the arrow sits one cell short of a gap-adjacent cell.
        public static int ExitDistanceIgnoringPiles(LevelDef level, Arrow a)
        {
            int x = a.X, y = a.Y, dir = a.Dir;
            for (int step = 1; step &lt;= 2; step++)
            {
                int nx = x + DX[dir], ny = y + DY[dir];
                bool off = (nx &lt; 0 || ny &lt; 0 || nx &gt;= level.W || ny &gt;= level.H);
                if (off)
                {
                    bool gap;
                    if (nx &gt;= level.W) gap = level.GapRight[y];
                    else if (nx &lt; 0) gap = level.GapLeft[y];
                    else if (ny &gt;= level.H) gap = level.GapBottom[x];
                    else gap = level.GapTop[x];
                    return gap ? step : -1;
                }
                if (level.Walls[ny * level.W + nx]) return -1;
                x = nx; y = ny;
            }
            return -1;
        }
    }
    public static class ArrowLevels
    {
        static LevelDef New(string name, int w, int h, int arrows)
        {
            LevelDef d = new LevelDef();
            d.Name = name; d.W = w; d.H = h;
            d.Walls = new bool[w * h];
            d.GapRight = new bool[h]; d.GapLeft = new bool[h];
            d.GapTop = new bool[w]; d.GapBottom = new bool[w];
            d.ArrowCount = arrows;
            d.Ax = new int[arrows]; d.Ay = new int[arrows]; d.Adir = new int[arrows];
            d.Atype = new ArrowType[arrows];
            return d;
        }
```

```
static void Set(LevelDef d, int id, int x, int y, int dir, ArrowType t)
{
    d.Ax[id] = x; d.Ay[id] = y; d.Adir[id] = dir; d.Atype[id] = t;
}
// L1 hook board 7x5: two base arrows on the exit lane. First tap: the back
// arrow slams into its buddy (comedy beat). Second tap: the front arrow
// escapes. Third tap: the buddy follows. Solutions: [0,1] and [1,0,1] = 2.
public static LevelDef Hook()
{
    LevelDef d = New("first_flight", 7, 5, 2);
    d.GapRight[2] = true;
    Set(d, 0, 5, 2, 0, ArrowType.Base);
    Set(d, 1, 0, 2, 0, ArrowType.Base);
    return d;
}
// L2 chain board 7x7: A (id 0) blocks the row-3 lane for B and the col-2 lane
// for C. B and C both pile-stop behind A; when A escapes, BOTH dependents
// unblock (chain depth 2). Ten distinct solution orders up to length 8.
public static LevelDef BuddyBlock()
{
    LevelDef d = New("buddy_block", 7, 7, 3);
    d.GapRight[3] = true; d.GapBottom[2] = true;
    Set(d, 0, 2, 3, 0, ArrowType.Base);
    Set(d, 1, 0, 3, 0, ArrowType.Base);
    Set(d, 2, 2, 0, 1, ArrowType.Base);
    return d;
}
// L3 trap board 5x5 (near-miss showcase, honestly unsolvable by design):
// B (id 2) always escapes; A (id 0) and C (id 1) end in a mutual block with A
// exactly one cell short of the top gap. Stuck -&gt; near-miss beat -&gt; optional
// rescue/skip offers -&gt; honest fail, zero punishment, dex keeps everything.
public static LevelDef OneCellShort()
{
    LevelDef d = New("one_cell_short", 5, 5, 3);
    d.GapTop[2] = true; d.GapBottom[2] = true; d.GapBottom[4] = true;
    Set(d, 0, 2, 4, 3, ArrowType.Base);
    Set(d, 1, 2, 0, 1, ArrowType.Base);
    Set(d, 2, 4, 0, 1, ArrowType.Base);
    return d;
}
// L4 type vocabulary 6x5: ghost glides through the wall, turner bounces
// clockwise into the bottom gap, double walks exactly two cells per ignition.
public static LevelDef TypeVocabulary()
{
    LevelDef d = New("type_vocabulary", 6, 5, 3);
    d.Walls[1 * 6 + 3] = true;   // (3,1)
    d.Walls[3 * 6 + 2] = true;   // (2,3)
    d.GapRight[1] = true; d.GapBottom[1] = true; d.GapLeft[4] = true;
    Set(d, 0, 0, 1, 0, ArrowType.Ghost);
    Set(d, 1, 0, 3, 0, ArrowType.Turner);
```

```
        Set(d, 2, 4, 4, 2, ArrowType.Double);
        return d;
    }
    // L5 dense board 8x6, thirteen arrows: V heads the column, then six rows of
    // back/front pairs. Solver first solution = 19 ignitions, zero backtracking.
    public static LevelDef ThirteenHighway()
    {
        LevelDef d = New("thirteen_highway", 8, 6, 13);
        for (int y = 0; y &lt; 6; y++) d.GapRight[y] = true;
        d.GapBottom[3] = true;
        Set(d, 0, 3, 0, 1, ArrowType.Base);
        for (int y = 0; y &lt; 6; y++)
        {
            Set(d, 1 + 2 * y, 2, y, 0, ArrowType.Base);
            Set(d, 2 + 2 * y, 5, y, 0, ArrowType.Base);
        }
        return d;
    }
    // L6 double-step board 5x4 (S2 ladder "sweet" tier): the double arrow walks
    // exactly two cells per ignition, so crossing the lane takes three taps - a
    // micro-rhythm beat. The up arrow shares column 2 with the double's first
    // stop, so an early up-tap piles it one cell short until the double clears
    // the midpoint; every ordering still solves (multi-solution face).
    public static LevelDef DoubleSteps()
    {
        LevelDef d = New("double_steps", 5, 4, 3);
        d.GapRight[1] = true; d.GapTop[2] = true;
        Set(d, 0, 0, 1, 0, ArrowType.Double);
        Set(d, 1, 3, 1, 0, ArrowType.Base);
        Set(d, 2, 2, 3, 3, ArrowType.Base);
        return d;
    }
    // L7 twist board 6x6 (turner vocabulary deepening): the turner slams into
    // the wall, twists clockwise once and dives down a fresh lane; the up
    // arrow standing on the twist column reroutes the dive one lane left, and
    // the down arrow parked on that lane forces a clear-order chain beat. Both
    // dive lanes have gaps, so every clearing order ends clean.
    public static LevelDef TwistLane()
    {
        LevelDef d = New("twist_lane", 6, 6, 3);
        d.Walls[0 * 6 + 3] = true;   // (3,0)
        d.GapTop[2] = true; d.GapBottom[1] = true; d.GapBottom[2] = true;
        Set(d, 0, 0, 0, 0, ArrowType.Turner);
        Set(d, 1, 2, 0, 3, ArrowType.Base);
        Set(d, 2, 1, 3, 1, ArrowType.Base);
        return d;
    }
    // L8 twin relay board 7x5 (S2 ladder "sweet" consolidation): two exit
    // lanes; the row-3 back arrow rides behind its front buddy (the L1
    // pile-stop beat grown into a lane pair) while the row-1 arrow runs
```

```
// free. Multi-solution face: seven distinct orders up to length 8 - the
// back arrow either waits for its buddy to leave or takes an extra
// stop-and-go lap behind it.
public static LevelDef TwinLanes()
{
    LevelDef d = New("twin_lanes", 7, 5, 3);
    d.GapRight[1] = true; d.GapRight[3] = true;
    Set(d, 0, 4, 1, 0, ArrowType.Base);
    Set(d, 1, 1, 3, 0, ArrowType.Base);
    Set(d, 2, 5, 3, 0, ArrowType.Base);
    return d;
}
// L9 ghost tunnel 6x6 (ghost vocabulary deepening): the ghost punches
// through the wall brick, but while the front arrow still holds the
// far side its single pass is spent - it parks one cell short and
// becomes a pile the down arrow must wait out. On a later ignition the
// pass refreshes and the ghost slides straight through its parked
// buddy. Chain depth 2 with the wall-pass + pile-pass vocabulary.
public static LevelDef GhostTunnel()
{
    LevelDef d = New("ghost_tunnel", 6, 6, 3);
    d.Walls[3 * 6 + 2] = true;   // (2,3)
    d.GapRight[3] = true; d.GapBottom[3] = true;
    Set(d, 0, 4, 3, 0, ArrowType.Base);
    Set(d, 1, 1, 3, 0, ArrowType.Ghost);
    Set(d, 2, 3, 0, 1, ArrowType.Base);
    return d;
}
// L10 moving wall 7x5 (double vocabulary deepening): the double hops
// exactly two cells per ignition, so its rest spots (2,1) (4,1) (6,1)
// march across row 1 like a sliding gate; each up arrow must climb
// around the gate's schedule or park one cell short and re-ignite once
// the gate has passed. First board where a moving obstacle - not a
// static pile - is the planning axis.
public static LevelDef MovingWall()
{
    LevelDef d = New("moving_wall", 7, 5, 3);
    d.GapRight[1] = true; d.GapTop[2] = true; d.GapTop[4] = true;
    Set(d, 0, 0, 1, 0, ArrowType.Double);
    Set(d, 1, 2, 3, 3, ArrowType.Base);
    Set(d, 2, 4, 3, 3, ArrowType.Base);
    return d;
}
// L11 turn gate 6x5 (S2 ladder sweet-&gt;tight transition, first combination
// board): the turner sweeps row 1, hits the wall brick and twists in place
// into its own column dive toward the bottom gap - but the base arrow
// guards the dive exit cell, so the dive stops one cell short and waits.
// The double's first rest spot sits exactly on the dive lane, so the pair
// must untangle in order: the double clears the lane or the turner parks
// below it, and an impatient re-ignition turns the diver sideways into a
```

```
// dead end (trap branch, honest fail vocabulary). Turner x double x
// pile-wait scheduling in one board.
public static LevelDef TurnGate()
{
    LevelDef d = New("turn_gate", 6, 5, 3);
    d.Walls[1 * 6 + 3] = true;   // (3,1)
    d.GapRight[3] = true; d.GapRight[4] = true; d.GapBottom[2] = true;
    Set(d, 0, 0, 1, 0, ArrowType.Turner);
    Set(d, 1, 0, 3, 0, ArrowType.Double);
    Set(d, 2, 2, 4, 0, ArrowType.Base);
    return d;
}
// L12 ghost relay 7x5 (chain deepening, depth 3): the ghost punches the
// wall brick but parks inside it one cell behind the front runner; a
// later ignition refreshes the pass and it slides straight through the
// parked buddy (wall-pass grown into a pile-pass). The climber below
// starts behind the ghost's own launch cell and must wait for it to
// leave - front -&gt; ghost -&gt; climber, a three-step dependency chain.
public static LevelDef GhostRelay()
{
    LevelDef d = New("ghost_relay", 7, 5, 3);
    d.Walls[2 * 7 + 4] = true;   // (4,2)
    d.GapRight[2] = true; d.GapTop[0] = true;
    Set(d, 0, 5, 2, 0, ArrowType.Base);
    Set(d, 1, 0, 2, 0, ArrowType.Ghost);
    Set(d, 2, 0, 4, 3, ArrowType.Base);
    return d;
}
// L13 gate relays 7x5 (moving-wall grown symmetric): the double hops
// (2,1) (4,1) (6,1) then exits - a sliding gate; one climber waits at
// its first rest column, the other at its last, and each must time the
// climb between the gate's beats or park one cell short and re-ignite
// once it has passed. Twin relay scheduling around a moving obstacle.
public static LevelDef GateRelays()
{
    LevelDef d = New("gate_relays", 7, 5, 3);
    d.GapRight[1] = true; d.GapTop[2] = true; d.GapTop[6] = true;
    Set(d, 0, 0, 1, 0, ArrowType.Double);
    Set(d, 1, 2, 3, 3, ArrowType.Base);
    Set(d, 2, 6, 3, 3, ArrowType.Base);
    return d;
}
// L14 column gate 6x6 (moving-wall verticalized, westward flow): the
// double hops DOWN column 2 - rest spots (2,2) (2,4) then out the
// bottom - a sliding gate standing upright. Two westward runners cross
// exactly the two rest rows, so an early runner glides under the
// untouched gate, but a runner that fires late slams into the parked
// double (comedy beat) and must wait for the gate to hop on. First
// board where the moving obstacle marches vertically and the crossers
// run against the reading direction.
```

```
public static LevelDef ColumnGate()
{
    LevelDef d = New("column_gate", 6, 6, 3);
    d.GapLeft[2] = true; d.GapLeft[4] = true; d.GapBottom[2] = true;
    Set(d, 0, 2, 0, 1, ArrowType.Double);
    Set(d, 1, 5, 2, 2, ArrowType.Base);
    Set(d, 2, 5, 4, 2, ArrowType.Base);
    return d;
}
// L15 conga line 7x6 (chain deepening, depth 4 - the deepest yet): the
// ghost parks inside the wall one cell behind the front runner; the
// first climber parks under the ghost's launch cell; the second
// climber cannot even move (a zero-tap shove is rejected) until the
// first climber clears out. Front -&gt; ghost -&gt; climber -&gt; climber: a
// four-step dependency conga. The ghost's re-ignition pass phases it
// straight through the parked front runner, so every respectful
// ordering untangles clean.
public static LevelDef CongaLine()
{
    LevelDef d = New("conga_line", 7, 6, 4);
    d.Walls[2 * 7 + 4] = true;   // (4,2)
    d.GapRight[2] = true; d.GapTop[0] = true;
    Set(d, 0, 5, 2, 0, ArrowType.Base);
    Set(d, 1, 0, 2, 0, ArrowType.Ghost);
    Set(d, 2, 0, 4, 3, ArrowType.Base);
    Set(d, 3, 0, 5, 3, ArrowType.Base);
    return d;
}
// L16 phase gate 7x5 (burst tier opener, two schedules braided): the
// double hops (2,1) (4,1) (6,1) then exits - the horizontal gate; the
// ghost must punch the wall brick and then wait behind the parked
// front runner (wall-pass grown into a pile-wait), while the climber
// must climb past the gate's first rest spot or park one cell short.
// Two independent waits on one board, and a curious branch: if the
// climber parks under the gate first, the ghost spends its pass on
// the CLIMBER pile instead of the wall and stalls one cell earlier.
public static LevelDef PhaseGate()
{
    LevelDef d = New("phase_gate", 7, 5, 4);
    d.Walls[2 * 7 + 4] = true;   // (4,2)
    d.GapRight[1] = true; d.GapRight[2] = true; d.GapTop[2] = true;
    Set(d, 0, 0, 1, 0, ArrowType.Double);
    Set(d, 1, 1, 2, 0, ArrowType.Ghost);
    Set(d, 2, 5, 2, 0, ArrowType.Base);
    Set(d, 3, 2, 3, 3, ArrowType.Base);
    return d;
}
// L17 bounce relay 7x6 (burst tier, pile-bounce vocabulary debut): the
// turner slams into its BUDDY pile - not a wall brick - twists clockwise
// into a vertical dive down its own stopped column, and either slips
```

```
// straight out the bottom door or parks one cell behind the diver
// already queuing there (comedy stop). Re-igniting the parked turner
// while the diver still holds the door twists it sideways into a dead
// lane (honest-fail branch), and letting the buddy leave before the
// bounce strands the turner against the right border (second
// honest-fail branch): the bounce is a one-time appointment. Diver and
// turner exit through the same bottom gap - a conga through one door -
// while the westward runner times the diver's start cell. First board
// whose redirect trigger is a living pile.
public static LevelDef BounceRelay()
{
    LevelDef d = New("bounce_relay", 7, 6, 4);
    d.GapTop[4] = true; d.GapBottom[3] = true; d.GapLeft[4] = true;
    Set(d, 0, 0, 2, 0, ArrowType.Turner);
    Set(d, 1, 4, 2, 3, ArrowType.Base);
    Set(d, 2, 3, 4, 1, ArrowType.Base);
    Set(d, 3, 6, 4, 2, ArrowType.Base);
    return d;
}
// L18 gate pass 7x6 (burst tier, pass-through-the-gate vocabulary): the
// double hops (4,2) (6,2) then exits - a two-rest gate; the ghost must
// punch the wall brick and then faces the gate itself: while the double
// is parked on the ghost's lane the ghost spends its single pass on the
// living pile and parks one cell below, waiting for a second ignition -
// but time the tap to the gate's schedule and the lane is clear, the
// pass is saved for the wall, one tap does both. The down arrow plays
// the same clock on the gate's far rest spot. First board where a
// moving obstacle eats the ghost's one pass.
public static LevelDef GatePass()
{
    LevelDef d = New("gate_pass", 7, 6, 3);
    d.Walls[3 * 7 + 4] = true;   // (4,3)
    d.GapRight[2] = true; d.GapTop[4] = true; d.GapBottom[6] = true;
    Set(d, 0, 2, 2, 0, ArrowType.Double);
    Set(d, 1, 4, 5, 3, ArrowType.Ghost);
    Set(d, 2, 6, 0, 1, ArrowType.Base);
    return d;
}
// L19 corner walk 7x6 (burst tier, staircase turner debut): the turner
// walks a two-leg staircase - slam the wall, twist down its own column,
// park on top of the second brick; ignite again, twist left, glide out
// the left door: the first arrow whose escape takes TWO twists in TWO
// taps, each ignition one leg. The runner leapfrogs behind the turner's
// parking cell, and the climber stops one cell short of the turner's
// launch cell - fire the walker's second leg into the waiting climber
// and the two pile up in a reversed dependency (climber frees the
// walker), but twist the walker again into the closed top and it
// strands (honest-fail branch). Two crossing arrows reading one
// walker's schedule.
public static LevelDef CornerWalk()
```

```
{
    LevelDef d = New("corner_walk", 7, 6, 3);
    d.Walls[1 * 7 + 3] = true;   // (3,1)
    d.Walls[3 * 7 + 2] = true;   // (2,3)
    d.GapLeft[2] = true; d.GapTop[0] = true;
    Set(d, 0, 0, 1, 0, ArrowType.Turner);
    Set(d, 1, 6, 2, 2, ArrowType.Base);
    Set(d, 2, 0, 5, 3, ArrowType.Base);
    return d;
}
// L20 double bounce 7x6 (burst tier, moving-appointment bounce debut): the
// double hops down column 4 - rest spots (4,2) then (4,4) - and the turner
// crossing row 2 can ONLY bounce off the double while it rests at (4,2):
// fire the turner too early or too late and it slides into the closed right
// border and strands forever (a one-time appointment). The bounced turner
// twists into a dive down its own stopped column and slips out the bottom
// door - unless the climber still stands in that column: then it parks on
// the climber's back (comedy stop) and its second ignition twists off the
// living pile and escapes left. The westward runner crosses the gate's
// second rest spot and the rescue lane: slam, wait, follow - and a runner
// parked on the gate's next rest spot makes the gate itself slam into ITS
// back. Four arrows reading one gate's schedule.
public static LevelDef DoubleBounce()
{
    LevelDef d = New("double_bounce", 7, 6, 4);
    d.GapTop[3] = true; d.GapBottom[3] = true; d.GapBottom[4] = true; d.GapLeft[4] = true;
    Set(d, 0, 4, 0, 1, ArrowType.Double);
    Set(d, 1, 0, 2, 0, ArrowType.Turner);
    Set(d, 2, 3, 5, 3, ArrowType.Base);
    Set(d, 3, 6, 4, 2, ArrowType.Base);
    return d;
}
// L21 cross gates 7x6 (burst tier, orthogonal braided gates): two doubles
// marching on perpendicular lanes - one hopping right along row 1, one
// hopping down column 2 - and every other arrow reads both schedules at
// once. The climber lives in the horizontal gate's own column: while the
// gate stands on its launch cell the climber parks one cell below its back
// and waits for the hop; the runner's row crosses the vertical gate's
// launch cell: an early run slams into the parked double (comedy beat) and
// waits for it to hop on. Every wait unblocks on the next gate beat - a
// pure rhythm board where the only punishment is impatience.
public static LevelDef CrossGates()
{
    LevelDef d = New("cross_gates", 7, 6, 4);
    d.GapRight[1] = true; d.GapBottom[2] = true; d.GapTop[4] = true; d.GapLeft[2] = true;
    Set(d, 0, 4, 1, 0, ArrowType.Double);
    Set(d, 1, 2, 2, 1, ArrowType.Double);
    Set(d, 2, 4, 5, 3, ArrowType.Base);
    Set(d, 3, 6, 2, 2, ArrowType.Base);
    return d;
```

```
}
// L22 ghost step 7x6 (burst tier, ghost-body bounce debut): the ghost
// punches the wall brick and parks INSIDE it, one cell behind the front
// runner - and its transparent body becomes the turner's stepping stone:
// the turner slams into the parked ghost, twists and dives a fresh column
// out the bottom door. Fire the turner before the ghost has moved and it
// bounces off the ghost's LAUNCH cell instead - a dive down the climber's
// column that parks on the climber's back and rescues itself out the left
// door on a second twist, or strands in the closed dead column if the
// climber has already gone (honest-fail branch). Later the ghost's pass
// refreshes and it slides straight through its parked buddy. Three bounce
// columns, three doors, one body.
public static LevelDef GhostStep()
{
    LevelDef d = New("ghost_step", 7, 6, 4);
    d.Walls[2 * 7 + 4] = true;   // (4,2)
    d.GapRight[2] = true; d.GapBottom[3] = true; d.GapTop[1] = true; d.GapLeft[4] = true;
    Set(d, 0, 0, 2, 0, ArrowType.Turner);
    Set(d, 1, 2, 2, 0, ArrowType.Ghost);
    Set(d, 2, 5, 2, 0, ArrowType.Base);
    Set(d, 3, 1, 5, 3, ArrowType.Base);
    return d;
}
// L23 gate conga 7x6 (burst tier, gate-led conga debut): the double hops
// DOWN column 2 - rest spot (2,4) then out the bottom door - and both
// followers queue above it inside its own lane. While the gate stands at
// (2,2) neither follower can even legally fire (a zero-tap shove straight
// into its back is rejected outright); the moment it hops to (2,4) the
// near follower advances to (2,3) - right on the runner's row - and the
// far follower steps up to (2,2). Every legal tap moves the tail exactly
// one station forward, and the eastward runner crossing row 3 either
// slips through ahead of the tail or slams into its parked back and
// waits for the whole conga to drain. Three arrows through one door,
// one station per beat: follow the leader.
public static LevelDef GateConga()
{
    LevelDef d = New("gate_conga", 7, 6, 4);
    d.GapBottom[2] = true; d.GapRight[3] = true;
    Set(d, 0, 2, 2, 1, ArrowType.Double);
    Set(d, 1, 2, 0, 1, ArrowType.Base);
    Set(d, 2, 2, 1, 1, ArrowType.Base);
    Set(d, 3, 0, 3, 0, ArrowType.Base);
    return d;
}
// L24 gate floor 7x6 (burst tier, moving-floor appointment): the turner
// slams the row-1 wall brick, twists in place and dives its OWN column -
// column 3, where the only floor waiting is the double's first rest spot
// (3,3). Keep the appointment and the dive parks on the gate's living
// back; leg two twists off the pile and glides out the row-2 left door.
// Miss it (fire before the gate arrives, or let the gate hop away
```

```
// between the legs) and the dive bottoms out at the closed border -
// every later ignition just walks the rim of the sealed board forever,
// a rat circling a locked room (honest-fail branch, no rim gap
// exists). The climber under the gate's second rest spot and the
// westward runner sharing the turner's own escape row read the same
// schedule: every arrow on the board answers to the gate.
public static LevelDef GateFloor()
{
    LevelDef d = New("gate_floor", 7, 6, 4);
    d.Walls[1 * 7 + 4] = true;   // (4,1)
    d.GapLeft[2] = true; d.GapRight[3] = true; d.GapTop[5] = true;
    Set(d, 0, 0, 1, 0, ArrowType.Turner);
    Set(d, 1, 1, 3, 0, ArrowType.Double);
    Set(d, 2, 5, 5, 3, ArrowType.Base);
    Set(d, 3, 6, 2, 2, ArrowType.Base);
    return d;
}
// L25 twin punch 7x6 (burst tier, two-brick corridor debut): two wall
// bricks fence row 3 and the ghost needs one ignition per brick - the
// twin punch demands exactly two passes. The diver at (3,3) is the cork:
// hold it in place and punch one parks the ghost INSIDE the first brick,
// one cell short of the cork's back; the second ignition phases it
// straight through the living cork and on to (5,3), the far station right
// before brick two. Drop the cork early and the very first punch walks
// the whole corridor in one breath, resting at the same far station -
// either way the next tap punches out. The climber under (5,3) waits
// beneath the puncher's parked body and climbs out only after the body
// has gone, while the hatch follower above the cork drops through the
// same floor door once the cork clears it. Every arrow reads the ghost's
// walk.
public static LevelDef TwinPunch()
{
    LevelDef d = New("twin_punch", 7, 6, 4);
    d.Walls[3 * 7 + 2] = true;   // (2,3)
    d.Walls[3 * 7 + 6] = true;   // (6,3)
    d.GapRight[3] = true; d.GapBottom[3] = true; d.GapTop[5] = true;
    Set(d, 0, 0, 3, 0, ArrowType.Ghost);
    Set(d, 1, 3, 3, 1, ArrowType.Base);
    Set(d, 2, 5, 5, 3, ArrowType.Base);
    Set(d, 3, 3, 1, 1, ArrowType.Base);
    return d;
}
// L26 leapfrog pair 7x6 (burst tier, lockstep doubles debut): two doubles
// share one lane and can only move in strict alternation - the rear double
// sits one cell behind the front one, so its first tap is a zero-move
// shove straight into its partner's back (rejected outright, comedy beat);
// once the front double hops, the rear one leapfrogs into the vacated
// station, and the pair crawls across row 1 two cells per beat. The
// interlock runs both ways: the front double's exit door is the rear one's
// last rest spot, so a late front hop seals the door and the rear double
```

```
// shoves the border (rejected) until its partner has fully gone. Meanwhile
// the westward turner slams the row-2 brick, twists up column 4 - straight
// through the rear double's first rest spot (4,1) - and out the top door;
// fire it while the double parks there and it stops one cell below,
// waiting for the train to pass. Fire it into the CLIMBER's parked back
// instead and the twist lands in column 5, which is sealed at the top:
// the turner strands at (5,0) forever (honest-fail branch). The climber
// under the same column reads both the turner's and the rear double's
// parking cells - every arrow on the board answers to the leapfrog.
public static LevelDef LeapfrogPair()
{
    LevelDef d = New("leapfrog_pair", 7, 6, 4);
    d.Walls[2 * 7 + 3] = true;   // (3,2)
    d.GapRight[1] = true; d.GapTop[4] = true;
    Set(d, 0, 2, 1, 0, ArrowType.Double);
    Set(d, 1, 3, 1, 0, ArrowType.Double);
    Set(d, 2, 6, 2, 2, ArrowType.Turner);
    Set(d, 3, 4, 5, 3, ArrowType.Base);
    return d;
}
// L27 drain gate 7x6 (burst tier, inverted conga debut - the leader
// waits): the double hops DOWN column 3 like the column gate, but this
// time the followers queue BELOW it, so the gate is the impatient one -
// its first hop parks it one cell behind the mid follower's back and
// every further tap is a zero-move shove (rejected) until the queue
// drains from the bottom. The low follower walks out first, the mid one
// follows (or parks one cell behind the low one and waits), and only
// then the gate crawls station by station down the emptied column. The
// westward runner crosses exactly the queue's mid-lane: an early run
// glides through the empty column, a late one slams into whichever
// arrow parks on (3,3) - follower or gate - and waits mid-board for the
// drain. The only punishment is impatience: every shove is rejected,
// nothing is ever lost, and every respectful order untangles clean.
public static LevelDef DrainGate()
{
    LevelDef d = New("drain_gate", 7, 6, 4);
    d.GapBottom[3] = true; d.GapLeft[3] = true;
    Set(d, 0, 3, 0, 1, ArrowType.Double);
    Set(d, 1, 3, 2, 1, ArrowType.Base);
    Set(d, 2, 3, 4, 1, ArrowType.Base);
    Set(d, 3, 6, 3, 2, ArrowType.Base);
    return d;
}
// L28 ghost stack 7x6 (burst tier, twin-ghost stack debut): TWO ghosts
// share the row-3 corridor and the single wall brick - the front one
// punches into the brick and parks inside it, one cell short of the
// cork runner holding the far station; the rear one then punches the
// SAME brick and phases into the very same cell - two transparent
// bodies stacked inside one brick, a ghost train in the wall. Fire the
// rear ghost first instead and it spends its single pass phasing
```

```
// through the front ghost's parked body and stops one cell short of the
// brick; fire the front one after that and it phases through the rear
// body - the twins swap places, pass for pass. The cork runner holds
// the corridor's far station until it walks out the right door; drop it
// too early and the first punch walks the whole corridor in one
// breath. The climber below the cork's column climbs out the top only
// after the cork has left its start cell - every arrow reads the
// cork's single step.
public static LevelDef GhostStack()
{
    LevelDef d = New("ghost_stack", 7, 6, 4);
    d.Walls[3 * 7 + 4] = true;   // (4,3)
    d.GapRight[3] = true; d.GapTop[5] = true;
    Set(d, 0, 1, 3, 0, ArrowType.Ghost);
    Set(d, 1, 0, 3, 0, ArrowType.Ghost);
    Set(d, 2, 5, 3, 0, ArrowType.Base);
    Set(d, 3, 5, 5, 3, ArrowType.Base);
    return d;
}
// L29 mirror cork 7x6 (finale tier, twin-turner debut): TWO turners share one
// mirror column - the west one bounces off the row-1 brick and dives column
// 3 down to the bottom door, the east one bounces off the row-4 brick and
// climbs the very same column up to the top door, and a cork base arrow
// sits at the column's heart (3,3). Whoever dives while the cork holds its
// seat stops one cell short and waits; the cork walks out the bottom door
// the moment its lane is clear, and then both mirrors flow like water. But
// the column has NO side doors: a parked turner's rescue twist (fresh turn
// on a new ignition) carries it to the sealed west or east edge and strands
// it there forever - respect the cork's schedule or lose the board, honest
// fail. The splay escape: when the east turner climbs into its twin's
// parked back it stops one cell short, and its rescue twist off the twin
// fires it out the clerk's east door instead - the only forgiveness the
// mirror column ever shows. The clerk base arrow crosses the cork's cell on
// row 3: fire it early and it pile-stops behind the cork, and every
// impatient second tap is a zero-move shove (rejected).
public static LevelDef MirrorCork()
{
    LevelDef d = New("mirror_cork", 7, 6, 4);
    d.Walls[1 * 7 + 4] = true;   // (4,1)
    d.Walls[4 * 7 + 2] = true;   // (2,4)
    d.GapBottom[3] = true; d.GapTop[3] = true; d.GapRight[3] = true;
    Set(d, 0, 1, 1, 0, ArrowType.Turner);
    Set(d, 1, 5, 4, 2, ArrowType.Turner);
    Set(d, 2, 3, 3, 1, ArrowType.Base);
    Set(d, 3, 0, 3, 0, ArrowType.Base);
    return d;
}
// L30 full house 7x6 (finale tier, the whole vocabulary on one board): the
// farewell board braids all four arrow families into a single schedule. The
// ghost's only door is through the row-2 brick - but the double's first
```

```
    // station (5,2) sits right beyond it, so a late ghost punches into the wall
    // and parks INSIDE the brick, a guest in the wall hotel, waiting for the
    // double to walk on (or phases out straight through its back on a fresh
    // pass). The double walks column 5 exactly two cells per tap - station
    // (5,2), station (5,4), then out the bottom door - and the clerk base
    // arrow pile-stops behind its second station and waits for the checkout.
    // The turner reads the ghost's schedule: bounce off the ghost's home body
    // and it dives its own column 1 out the bottom door; catch the lane empty
    // (ghost gone or in-wall) and it runs to the brick, twists at (3,2) and
    // dives its own column 3 instead - same mirror column as the grand finale,
    // same law: the blocker ahead of a turner decides nothing, the cell it
    // STANDS on decides everything. Every family answers to another: ghost to
    // the double's station, double to the bottom door, clerk to the double's
    // checkout, turner to the ghost's timing. The full house, packed and gone.
    public static LevelDef FullHouse()
    {
        LevelDef d = New("full_house", 7, 6, 4);
        d.Walls[2 * 7 + 4] = true;   // (4,2)
        d.GapRight[2] = true; d.GapBottom[5] = true; d.GapBottom[1] = true;
        d.GapBottom[3] = true; d.GapRight[4] = true;
        Set(d, 0, 2, 2, 0, ArrowType.Ghost);
        Set(d, 1, 5, 0, 1, ArrowType.Double);
        Set(d, 2, 0, 2, 0, ArrowType.Turner);
        Set(d, 3, 0, 4, 0, ArrowType.Base);
        return d;
    }
}
public sealed class GrayboxSim
{
    // ---- Stage1 hook timeline (frozen at E4 authoring - zero loosening) ----
    public const double CaptionT = 0.15;        // hook caption (red line 11, T+0-3s)
    public const double LevelLoadT = 0.50;     // first board lands
    public const double FirstIgniteT = 1.00;   // first tap: the arrow takes off alone
    public const double SecondIgniteT = 2.50;  // front arrow escapes through the gap
    public const double ReIgniteT = 3.50;     // the blocked buddy follows it out
    public const double MoveSecondsPerCell = 0.25;
    // ---- guardrail vocabulary bans (E1 admission compliance face) ----
    public static readonly string[] ForbiddenTokens =
    {
        "arrows_puzzle_escape", "miniclip", "wiggle_escape", "arrow_rush"
    };
    public static readonly string[] PunishTokens =
    {
        "punish", "penalty", "wipe", "reset_progress", "streak_break", "confiscate"
    };
    public static readonly string[] ShareFormTokens = { "score", "points", "rank" };
    public static int RngCalls = 0;                    // stays 0 forever
    public const bool AdsRequiredForClosure = false;   // ads purely optional
    public double SimTime;
    public readonly List&lt;SimEvent&gt; Events = new List&lt;SimEvent&gt;();
```

```
public LevelDef Level;
public Arrow[] Arrows;
public int IgniteAttempts, IgniteLegal, IgniteRejects, ArrowsOut;
public bool LevelComplete, LevelFailed, StuckRaised;
public int NearMisses, RescueOffers, ShareCards;
public int LevelsSeen, LevelsCompleted;             // dex: fail never wipes
public int GrantsHint, GrantsRescue, GrantsSkip, AdsWatched;
public CodexCore Codex;                     // S2 T013 codex layer (optional: null = zero drift)
public bool GrantsTainted;                  // per-board: any optional grant used since load
public void Log(string type, string detail) { Events.Add(new SimEvent(SimTime, type, detail)); }
public double TimeOf(string type)
{
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) return Events[i].t;
    return -1.0;
}
public bool EventExistsAt(string type, double t)
{
    for (int i = 0; i &lt; Events.Count; i++)
        if (Events[i].type == type &amp;&amp; Math.Abs(Events[i].t - t) &lt; 1e-9) return true;
    return false;
}
public void Caption(string text) { Log("caption", text); }
public void LoadLevel(LevelDef def)
{
    Level = def;
    LevelsSeen++;
    GrantsTainted = false;   // fresh board: the no-rescue star watches this board only
    LevelComplete = false; LevelFailed = false; StuckRaised = false;
    ArrowsOut = 0;
    Arrows = new Arrow[def.ArrowCount];
    for (int i = 0; i &lt; def.ArrowCount; i++)
    {
        Arrows[i] = new Arrow();
        Arrows[i].Id = i; Arrows[i].X = def.Ax[i]; Arrows[i].Y = def.Ay[i];
        Arrows[i].Dir = def.Adir[i]; Arrows[i].Type = def.Atype[i]; Arrows[i].OnBoard = true;
    }
    Log("level_load", "board loaded: " + def.Name + " (" + def.ArrowCount + " arrows, " + def.W + "x" + def.H +
}
public bool Ignite(int id)
{
    IgniteAttempts++;
    SlideResult r = BoardOps.ComputeSlide(Level, Arrows, id);
    if (!r.Legal)
    {
        string why = r.OutOfRange ? "out of range" : r.AlreadyOut ? "arrow already out" : "slide blocked - no ro
        Log("ignite_rejected", "arrow " + id + " rejected: " + why);
        IgniteRejects++;
        return false;
    }
    IgniteLegal++;
```

```
    Arrow a = Arrows[id];
    if (Codex != null) Codex.OnArrowIgnited(a.Type);   // S2 T013: first legal ignite joins the arrow row
    Log("player_tap", "tap arrow " + id + " (" + BoardOps.TypeName(a.Type) + " " + BoardOps.DirName(a.Dir) + ")"
    Log("sfx_ignite", "arrow " + id + " takes off on its own");
    Log("slide_start", "arrow " + id + " slides by itself");
    if (r.GhostPass) Log("ghost_pass", "arrow " + id + " glides straight through the wall");
    if (r.TurnFired) Log("turn_fire", "arrow " + id + " hits a block and turns clockwise");
    if (r.Exited)
    {
        a.OnBoard = false; a.X = -1; a.Y = -1;
        ArrowsOut++;
        double t = SimTime + r.Moves * MoveSecondsPerCell;
        Events.Add(new SimEvent(t, "arrow_out",
            "arrow " + id + " escapes through the gap after " + r.Moves + " cells - gone"));
    }
    else
    {
        a.X = r.X; a.Y = r.Y; a.Dir = r.Dir;
        double t = SimTime + r.Moves * MoveSecondsPerCell;
        string kind = r.BlockedByPileId &gt;= 0 ? ("pile " + r.BlockedByPileId)
            : (r.DoubleStop ? "its own two-cell limit" : "a wall");
        Events.Add(new SimEvent(t, "slide_stop",
            "arrow " + id + " slams into " + kind + " after " + r.Moves + " cells"));
        if (r.BlockedByPileId &gt;= 0)
            Events.Add(new SimEvent(t, "pile_stop",
                "arrow " + id + " blocked by pile " + r.BlockedByPileId + " after " + r.Moves + " cells"));
        if (r.DoubleStop)
            Events.Add(new SimEvent(t, "double_stop",
                "arrow " + id + " stops after exactly " + r.Moves + " cells (double)"));
    }
    CheckAllOff();
    return true;
}
void CheckAllOff()
{
    if (LevelComplete || LevelFailed) return;
    for (int i = 0; i &lt; Arrows.Length; i++) if (Arrows[i].OnBoard) return;
    LevelComplete = true;
    LevelsCompleted++;
    if (Codex != null) Codex.OnBoardCleared(Level.Name, GrantsTainted);   // S2 T013 codex wiring
    Log("level_complete", "board clear - every arrow is out");
    ShareCard();
}
// stuck detection: no idle arrow has a legal slide. On a stuck board, an idle
// arrow sitting one cell short of a gap fires the near-miss beat plus the two
// optional IAA outs (rescue / skip). One beat per stuck board.
public bool CheckStuck()
{
    int idle = 0;
    for (int i = 0; i &lt; Arrows.Length; i++)
```

```
        {
            if (!Arrows[i].OnBoard) continue;
            idle++;
            if (BoardOps.ComputeSlide(Level, Arrows, i).Legal) return false;
        }
        if (idle == 0) return false;
        StuckRaised = true;
        Log("level_stuck", "no legal move remains - " + idle + " arrows still on the board");
        for (int i = 0; i &lt; Arrows.Length; i++)
        {
            Arrow a = Arrows[i];
            if (!a.OnBoard) continue;
            if (BoardOps.ExitDistanceIgnoringPiles(Level, a) == 2)
            {
                Arrow blk = BoardOps.PileAt(Level, Arrows, a.Id,
                    a.X + BoardOps.DX[a.Dir], a.Y + BoardOps.DY[a.Dir]);
                string who = blk != null ? ("blocked by pile " + blk.Id) : "blocked just short";
                NearMisses++;
                Log("near_miss", "arrow " + a.Id + " is one cell short of the gap, " + who + " (so close)");
                Log("rescue_offer", "optional video: remove the blocking pile? (rescue)");
                Log("rescue_offer", "optional video: skip this board? (skip)");
                RescueOffers += 2;
                break;
            }
        }
        return true;
    }
    public void DeclineRescue()
    {
        Log("rescue_declined", "the player passes on every optional video - the board moves on");
    }
    public void FailLevel()
    {
        LevelFailed = true;
        Log("level_fail", "the board stays unsolved - nothing is lost, the level dex keeps everything");
    }
    // Share face (B904 guardrail #2 analog): board-clear achievement card with an
    // X032-style code. Score-form display stays out of the payload.
    public void ShareCard()
    {
        ShareCards++;
        Log("share_card", "board clear achievement card code=AO1-"
            + LevelsCompleted.ToString("X", CultureInfo.InvariantCulture)
            + "-" + Arrows.Length + " (all arrows out)");
    }
    // ---- IAA stub: three optional rewarded-video positions (S1 wiring face) ----
    public string GrantHint()
    {
        GrantsHint++; AdsWatched++; GrantsTainted = true;
        string hint = GrayboxBot.SolveFirstMove(Level, Arrows);
```

```
        Log("iaa_grant", "hint granted (rewarded video): " + (hint != null ? hint : "no move helps"));
        return hint;
    }
    public void GrantRescue(int pileId)
    {
        GrantsRescue++; AdsWatched++; GrantsTainted = true;
        if (pileId &lt; 0 || pileId &gt;= Arrows.Length || !Arrows[pileId].OnBoard) return;
        Arrows[pileId].OnBoard = false;
        Arrows[pileId].X = -1; Arrows[pileId].Y = -1;
        Log("iaa_grant", "rescue granted (rewarded video): pile " + pileId + " removed - the lane opens");
        CheckAllOff();
    }
    public void GrantSkip()
    {
        GrantsSkip++; AdsWatched++; GrantsTainted = true;
        Log("iaa_grant", "skip granted (rewarded video): the rest of the board auto-clears");
        List&lt;int&gt; order = GrayboxBot.SolveFromSnapshot(Level, Arrows);
        if (order != null)
        {
            for (int i = 0; i &lt; order.Count; i++) { SimTime += 0.5; Ignite(order[i]); }
        }
    }
    // ---- Stage3 dense board (perf probe only) ----
    public readonly int[] ArrowStream = new int[2000];
    public readonly double[] LaneTicker = new double[512];
    public readonly double[] GlowField = new double[1024];
    public void BuildDense()
    {
        for (int i = 0; i &lt; 2000; i++) ArrowStream[i] = (i * 7) % 13;
        for (int m = 0; m &lt; 512; m++) LaneTicker[m] = ((m * 11) % 19) * 0.05;
        for (int g = 0; g &lt; 1024; g++) GlowField[g] = ((g * 5) % 23) * 0.05;
    }
    // Dense resolve: counting pass over the 2000-cell stream, contiguous-lane
    // window enumeration, then ticker/glow smoothing passes.
    public bool DenseResolve(int idx)
    {
        int[] cnt = new int[13];
        for (int i = 0; i &lt; 2000; i++) cnt[ArrowStream[i] % 13]++;
        long laneMult = 1L &lt;&lt; (6 + (idx % 7));
        long best = 0;
        for (int len = 1; len &lt;= 13; len++)
        {
            for (int start = 0; start + len - 1 &lt; 13; start++)
            {
                bool ok = true;
                for (int p = start; p &lt; start + len; p++) if (cnt[p] &lt; 1) { ok = false; break; }
                if (ok) { long s = len * laneMult; if (s &gt; best) best = s; }
            }
        }
        for (int m = 0; m &lt; 512; m++) LaneTicker[m] = (LaneTicker[m] + LaneTicker[(m * 31 + 7) % 512]) * 0.5;
```

```
            for (int g = 0; g &lt; 1024; g++) GlowField[g] = (GlowField[g] + GlowField[(g * 17 + 3) % 1024]) * 0.5;
            return best &gt; 0;
        }
        // Raw idle tick: stream drift + ticker relax + glow relax.
        public void DenseTick()
        {
            for (int i = 0; i &lt; 2000; i++) ArrowStream[i] = (ArrowStream[i] + 1) % 13;
            for (int m = 0; m &lt; 512; m++) LaneTicker[m] = (LaneTicker[m] + LaneTicker[(m * 31 + 7) % 512]) * 0.5;
            for (int g = 0; g &lt; 1024; g++) GlowField[g] = (GlowField[g] + GlowField[(g * 17 + 3) % 1024]) * 0.5;
        }
    }
}
// LocaleService.cs
// LocaleService.cs - S1 T010 c1 caption-key localization core for ArrowOut (B1833,
// G26 ExileFarm LocaleService pattern token-ported per the B1276 verbatim law).
// The pages host resolves every player-visible string through ByKey(CaptionKey)
// against the shipped cfg_i18n table (Resources/cfg_i18n.json, the ConfigValidator
// closed set - the validator mirror and this census stay in sync under the X1004
// twin law). Pure C#: the host parses the TextAsset with JsonUtility and injects
// the rows; the strict faces mirror the kit ctor law (duplicate keys reject,
// unknown keys reject) so a missing key fails loud at gate time, never silently
// at play time. The {N} slot face serves parameterized rows. Default language is
// zh (the Chinese-facing vertical); SetLang switches the ByKey face for the en
// mirror path (the frozen mirrors in PagesGate pin byte-equal to the en rows).
using System;
using System.Collections.Generic;
namespace Sy.ArrowOut
{
    public sealed class LocaleService
    {
        public const string DefaultLang = "zh";
        public const string EnLang = "en";
        readonly Dictionary&lt;string, string&gt; _en = new Dictionary&lt;string, string&gt;();
        readonly Dictionary&lt;string, string&gt; _zh = new Dictionary&lt;string, string&gt;();
        public string Lang = DefaultLang;
        public LocaleService(IList&lt;string&gt; keys, IList&lt;string&gt; en, IList&lt;string&gt; zh)
        {
            if (keys == null || en == null || zh == null)
                throw new ArgumentException("locale table rows must not be null");
            if (keys.Count != en.Count || keys.Count != zh.Count)
                throw new ArgumentException("locale table must carry en/zh for every key");
            for (int i = 0; i &lt; keys.Count; i++)
            {
                if (keys[i] == null || keys[i].Length == 0)
                    throw new ArgumentException("locale key " + i + " must be a CaptionKey");
                if (_en.ContainsKey(keys[i]))
                    throw new ArgumentException("duplicate locale key: " + keys[i]);
                if (en[i] == null || en[i].Length == 0 || zh[i] == null || zh[i].Length == 0)
                    throw new ArgumentException("locale key " + keys[i] + " must carry both en and zh copy");
                _en[keys[i]] = en[i];
```

```
            _zh[keys[i]] = zh[i];
        }
    }
    public int KeyCount { get { return _en.Count; } }
    public bool HasKey(string key) { return _en.ContainsKey(key); }
    /// &lt;summary&gt;Resolve copy by CaptionKey in the current language. Strict:
    /// an unknown key throws (fail loud at gate time - the closed-set census
    /// gate guarantees every referenced key exists).&lt;/summary&gt;
    public string ByKey(string key)
    {
        return ByKey(Lang, key);
    }
    /// &lt;summary&gt;Resolve copy by CaptionKey in an explicit language.&lt;/summary&gt;
    public string ByKey(string lang, string key)
    {
        if (key == null) throw new ArgumentException("CaptionKey must not be null");
        Dictionary&lt;string, string&gt; table = lang == EnLang ? _en : _zh;
        string value;
        if (!table.TryGetValue(key, out value))
            throw new ArgumentException("unknown CaptionKey: " + key);
        return value;
    }
    /// &lt;summary&gt;{N} slot face: replace the literal slot token with the value.
    /// Returns the input untouched when the slot is absent.&lt;/summary&gt;
    public static string ApplySlot(string text, string slot, string value)
    {
        if (text == null || slot == null) return text;
        return text.Replace("{" + slot + "}", value);
    }
}
/// &lt;summary&gt;Copy pool face (T014): the near_miss and share families ship as
/// variant pools inside the closed cfg_i18n key set (38 keys); the picker is
/// a pure function of the pool and the caller's counter (the sim event
/// count), so consecutive boards cycle the variants deterministically with
/// zero immediate repeat and zero RNG.&lt;/summary&gt;
public static class CopyPools
{
    public static readonly string[] NearMissKeys =
    {
        "near_miss_line", "near_miss_line_2", "near_miss_line_3", "near_miss_line_4"
    };
    public static readonly string[] ShareTitleKeys =
    {
        "share_card_title", "share_card_title_2", "share_card_title_3"
    };
    public static readonly string[] ShareBodyKeys =
    {
        "share_card_body", "share_card_body_2", "share_card_body_3"
    };
    /// &lt;summary&gt;Deterministic pool pick: the counter is the caller's event
```

```
        /// count; the modulo cycles the pool (negative counters fold in so the
        /// face stays total).&lt;/summary&gt;
        public static string Pick(string[] pool, int counter)
        {
            int n = pool.Length;
            int idx = ((counter % n) + n) % n;
            return pool[idx];
        }
    }
}
// PagesGate.cs
// PagesGate.cs - S1 T010 pages gate for ArrowOut (B1833, G26 ExileFarm PagesGate
// pattern ported onto the G28 frozen faces; kit-anchored teach beats + save face).
// Three acceptance faces from the taskbook row T010 (c1 slice): the frozen page
// list (S99_Boot / S01_Board / S02_Codex / S03_Share - zero missing, zero extra,
// every canvas carries a U184 scaler + raycaster, built by the view host in
// PagesHost.cs), the five-beat teach table (demand-doc section 3, frozen at E4
// authoring: 0.15 hook caption / 1.00 first ignite with the tap+sfx+slide_start
// triple / 2.00 the self-slide slam-stop that stakes the arrow into a pile /
// 3.00 the first escape / 3.50 board clear + the AO1- card), and the save face
// (levels completed + levels seen + share cards + ad grant stamps - strict
// compose-parse round-trip through ISaveService; the AdLedger ExportStamps IS
// the save.adcaps data face wired here per the T008 contract).
// The teach drive replays the canonical L1 hook opening on the real kit sim:
// 0.15 caption, 0.50 board load, 1.00 ignite the back arrow (it slides 4 cells
// and slams into its buddy -&gt; slide_stop + pile_stop at 2.00), 2.50 ignite the
// front arrow (2 cells out -&gt; arrow_out at 3.00), 3.50 the buddy follows it out
// (3 cells) and the board clears at the ignition tick - level_complete plus the
// AO1-1-2 board-clear card. Pure C# (no UnityEngine) so Editor gates drive it
// headless and the view host consumes the same frozen faces. Every number is a
// frozen anchor (E4 kit / demand-doc section 3, measured against the shipped kit
// by the b1833 dry-probe) - zero re-tuning, zero RNG.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
using Sy.Core;
namespace Sy.ArrowOut
{
    /// &lt;summary&gt;
    /// Frozen page registry (demand-doc section 3 page list) and teach anchors.
    /// Zero re-tuning: S2+ widens flavor, never the page set, the cell counts or
    /// the beat times.
    /// &lt;/summary&gt;
    public static class PageFrozen
    {
        public static readonly string[] Names = { "S99_Boot", "S01_Board", "S02_Codex", "S03_Share" };
        // L1 hook board frozen geometry (E4 ArrowLevels.Hook() face, the opening
        // board the teach drive replays and the board page renders at boot):
        // 7x5 grid, two base arrows on the exit lane, one right-edge gap.
```

```
    public const int BoardW = 7;
    public const int BoardH = 5;
    public const int BoardCells = BoardW * BoardH;   // 35 grid cells
    public const int BoardArrows = 2;                 // L1 ArrowCount face
    public const int BoardGaps = 1;                   // L1 GapRight[2] face
    // teach slide faces (measured against the shipped kit by the b1833
    // dry-probe, E4 L1 geometry): the back arrow slides 4 cells before it
    // slams into its buddy; the front arrow exits in 2 cells; the buddy's
    // re-ignition clears in 3 cells.
    public const int TeachSlideStopCells = 4;
    public const int TeachFirstEscapeCells = 2;
    public const int TeachBuddyExitCells = 3;
    // codex wall faces: the five frozen boards + the four arrow types.
    public const int CodexBoards = 5;
    public const int CodexTypes = 4;
    public const string SaveSlot = "arrowout_save";
    // U184 per-canvas law (the view host consumes these):
    public const int RefWidth = 1080;
    public const int RefHeight = 1920;
    public static bool IsPage(string name)
    {
        for (int i = 0; i &lt; Names.Length; i++) if (Names[i] == name) return true;
        return false;
    }
}
/// &lt;summary&gt;One red-line-11 teach beat: frozen time + id + page + i18n caption
/// key + the frozen English mirror of the key (the probe/anchor face; the host
/// resolves the player-visible copy through LocaleService by CaptionKey) + the
/// event anchor the beat teaches. Pure data, zero logic.&lt;/summary&gt;
public sealed class TeachBeat
{
    public readonly double T;
    public readonly string Id;
    public readonly string Page;
    public readonly string CaptionKey;
    public readonly string CaptionMirror;
    public readonly string Anchor;
    public TeachBeat(double t, string id, string page, string captionKey, string captionMirror, string anchor)
    {
        T = t; Id = id; Page = page; CaptionKey = captionKey; CaptionMirror = captionMirror; Anchor = anchor;
    }
}
public static class PagesGate
{
    // ---- the frozen five-beat table (demand-doc section 3, E4 authoring).
    // Three beats bind byte-equal onto the shipped kit constants; the 2.00
    // slam-stop and the 3.00 first escape bind onto the kit constants plus
    // the measured L1 slide distances. Monotonic by construction.
    public static readonly TeachBeat[] TeachBeats =
    {
```

```
        new TeachBeat(GrayboxSim.CaptionT, "hook",
            "S01_Board", "caption_hook",
            "One tap - the arrow walks itself.", "caption"),
        new TeachBeat(GrayboxSim.FirstIgniteT, "first_ignite",
            "S01_Board", "teach_first_ignite",
            "First ignition - tap once, the arrow slides on its own.", "player_tap+sfx_ignite+slide_start"),
        new TeachBeat(GrayboxSim.FirstIgniteT + PageFrozen.TeachSlideStopCells * GrayboxSim.MoveSecondsPerCell,
            "pile_stop", "S01_Board", "pile_stop_line",
            "Slammed and staked - the board just changed.", "slide_stop+pile_stop"),
        new TeachBeat(GrayboxSim.SecondIgniteT + PageFrozen.TeachFirstEscapeCells * GrayboxSim.MoveSecondsPerCell,
            "first_escape", "S01_Board", "arrow_out_line",
            "Out through the gap!", "arrow_out"),
        new TeachBeat(GrayboxSim.ReIgniteT, "board_clear",
            "S01_Board", "board_clear_line",
            "Board clear - every arrow is out.", "level_complete+share_card"),
    };
    // ---- the canonical L1 teach opening, replayed on the real kit sim with
    // the frozen beat times (the host paces the same beats in wall time; the
    // gate replays them in logical time). One code path for both.
    public static void RunTeachDrive(GrayboxSim sim)
    {
        if (sim == null) return;
        for (int i = 0; i &lt; TeachBeats.Length; i++) RunTeachBeat(sim, i);
    }
    /// &lt;summary&gt;One beat of the frozen five-beat opening on the real kit sim.
    /// Beat 2 is a display beat (the slam-stop events land at 2.00 from the
    /// 1.00 ignition) that also fires the 2.50 front-arrow ignition; beat 3
    /// is a pure display beat (the escape event lands at 3.00).&lt;/summary&gt;
    public static void RunTeachBeat(GrayboxSim sim, int beatIndex)
    {
        switch (beatIndex)
        {
            case 0:   // 0.15 hook caption
                sim.SimTime = GrayboxSim.CaptionT;
                sim.Caption(TeachBeats[0].CaptionMirror);
                break;
            case 1:   // 0.50 board load, 1.00 first ignite (the triple beat)
                sim.SimTime = GrayboxSim.LevelLoadT;
                sim.LoadLevel(ArrowLevels.Hook());
                sim.SimTime = GrayboxSim.FirstIgniteT;
                sim.Ignite(1);
                break;
            case 2:   // 2.00 slam-stop lands; 2.50 the front arrow escapes
                sim.SimTime = GrayboxSim.SecondIgniteT;
                sim.Ignite(0);
                break;
            case 3:   // 3.00 first escape lands - pure display beat
                break;
            case 4:   // 3.50 the buddy follows it out - board clear + card
                sim.SimTime = GrayboxSim.ReIgniteT;
```

```
            sim.Ignite(1);
            break;
    }
}
// ---- save face: level progress + dex + adcaps -------------------------
// Service round-trip helpers: the runtime asmdef carries the Sy.Core
// service face (the editor asmdef does not reference it), so the editor
// gate drives the ISaveService round-trip through these two helpers.
public static bool SaveFaceThroughService(SaveFace face)
{
    ServiceLocator.Get&lt;ISaveService&gt;().Save(PageFrozen.SaveSlot, SaveCodec.Compose(face));
    return true;
}
public static string LoadFacePayload()
{
    return ServiceLocator.Get&lt;ISaveService&gt;().Load(PageFrozen.SaveSlot);
}
/// &lt;summary&gt;The persisted face (demand-doc section 2 S07): the level
/// progress (completed boards), the dex (levels seen - never wiped on
/// loss, zero-punishment law - plus the share-card count), and the ad
/// grant stamps (the T008 AdLedger ExportStamps save.adcaps data face,
/// wall-clock seconds).&lt;/summary&gt;
public sealed class SaveFace
{
    public int LevelsCompleted;
    public int LevelsSeen;
    public int ShareCards;
    public readonly List&lt;long&gt; AdGrantStamps = new List&lt;long&gt;();
}
/// &lt;summary&gt;
/// Strict save codec (ShareCodec twin law): fixed prefix, fixed section
/// order, strict token grammar, the seen-implies-completed semantic law,
/// and a canonical compose so Compose(Parse(Compose(x))) == Compose(x)
/// always holds. Format (pure ASCII, zero hard caps vocabulary):
///   AOSAVE1|L=4|S=5|C=4|A=3600,7200
/// The grant stamps keep ledger order.
/// &lt;/summary&gt;
public static class SaveCodec
{
    public const string Prefix = "AOSAVE1";
    public static string Compose(SaveFace face)
    {
        StringBuilder sb = new StringBuilder();
        sb.Append(Prefix);
        sb.Append("|L=").Append(face.LevelsCompleted.ToString(CultureInfo.InvariantCulture));
        sb.Append("|S=").Append(face.LevelsSeen.ToString(CultureInfo.InvariantCulture));
        sb.Append("|C=").Append(face.ShareCards.ToString(CultureInfo.InvariantCulture));
        sb.Append("|A=");
        for (int i = 0; i &lt; face.AdGrantStamps.Count; i++)
        {
```

```
                if (i &gt; 0) sb.Append(',');
                sb.Append(face.AdGrantStamps[i].ToString(CultureInfo.InvariantCulture));
            }
            return sb.ToString();
        }
        public static bool TryParse(string payload, out SaveFace face)
        {
            face = null;
            if (payload == null) return false;
            string[] sections = payload.Split('|');
            if (sections.Length != 5 || sections[0] != Prefix) return false;
            if (!sections[1].StartsWith("L=", StringComparison.Ordinal) ||
                !sections[2].StartsWith("S=", StringComparison.Ordinal) ||
                !sections[3].StartsWith("C=", StringComparison.Ordinal) ||
                !sections[4].StartsWith("A=", StringComparison.Ordinal)) return false;
            SaveFace f = new SaveFace();
            if (!int.TryParse(sections[1].Substring(2), NumberStyles.None, CultureInfo.InvariantCulture, out f.Level
                f.LevelsCompleted &lt; 0) return false;
            if (!int.TryParse(sections[2].Substring(2), NumberStyles.None, CultureInfo.InvariantCulture, out f.Level
                f.LevelsSeen &lt; 0) return false;
            if (!int.TryParse(sections[3].Substring(2), NumberStyles.None, CultureInfo.InvariantCulture, out f.Share
                f.ShareCards &lt; 0) return false;
            if (f.LevelsSeen &lt; f.LevelsCompleted) return false;   // seen implies completed
            string a = sections[4].Substring(2);
            if (a.Length &gt; 0)
            {
                string[] stamps = a.Split(',');
                long ts;
                for (int i = 0; i &lt; stamps.Length; i++)
                {
                    if (!long.TryParse(stamps[i], NumberStyles.None, CultureInfo.InvariantCulture, out ts) ||
                        ts &lt; 0) return false;
                    f.AdGrantStamps.Add(ts);
                }
            }
            face = f;
            return true;
        }
    }
}
}
// PagesHost.cs
// PagesHost.cs - S1 T010 procedural UGUI pages host for ArrowOut (B1833, G26
// ExileFarm PagesHost pattern ported onto the G28 frozen pages, X027 paradigm:
// zero prefabs, zero scene serialization - the main scene only carries the camera,
// the shell canvas and the EventSystem; every page is built in code at runtime).
// Four frozen pages (PageFrozen registry): S99_Boot splash, S01_Board (board bg +
// the L1 opening board grid + the two arrow glyphs + the exit gap + the HUD strip
// codex/share/undo + the level label + the teach caption seat), S02_Codex (the
// five-board type wall + the four arrow-type entries), S03_Share (share frame +
```

```
// the AO1- code label + the board-clear card face).
// U184 per-canvas law: every page canvas carries a CanvasScaler locked to
// ScaleWithScreenSize at the frozen 1080x1920 portrait reference plus a
// GraphicRaycaster. The ProjectSetup placeholder canvas (the "S28_Main Main"
// label shell) is absorbed on build - placeholder shell law, zero stray canvas.
// Teach pacing walks the frozen PagesGate five-beat table in wall time; captions
// resolve through LocaleService by CaptionKey (zero hardcoded copy - every
// player-visible label reads from the shipped cfg_i18n table) and every beat
// drives the same canonical L1 opening on the real kit sim (the b1833 dry-probe
// timeline). Save wiring mirrors the level progress + the dex counters into the
// SaveFace and lands it through ISaveService (the save.adcaps data face = the
// AdLedger ExportStamps, the T008 contract).
// T014 slice2 copy pools: the near_miss and share families ship as cfg_i18n
// variant pools - the host consumes the sim events and fills the caption seat
// / the share card seats by CaptionKey through CopyPools with the event-count
// pick counter (deterministic cycle, zero RNG, zero hardcoded copy).
using System;
using System.Collections;
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.UI;
namespace Sy.ArrowOut
{
    public sealed class PagesHost : MonoBehaviour
    {
        static Font _font;
        static LocaleService _locale;
        readonly Dictionary&lt;string, GameObject&gt; _pages = new Dictionary&lt;string, GameObject&gt;();
        GrayboxSim _sim;
        AdLedger _ledger;
        int _nearMissPicks;   // near_miss pool pick counter (== the sim's near_miss event count)
        int _sharePicks;      // share pool pick counter (== the sim's share_card event count)
        public static Font SharedFont
        {
            get
            {
                if (_font == null) _font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
                return _font;
            }
        }
        [Serializable] class I18nRow { public string key; public string en; public string zh; }
        [Serializable] class I18nTable { public List&lt;I18nRow&gt; strings; }
        /// &lt;summary&gt;Locale face (T010 c1): parse the shipped cfg_i18n table (the
        /// ConfigValidator closed set) once, serve ByKey(CaptionKey) lookups - zh
        /// default, the Chinese-facing vertical. The strict LocaleService ctor
        /// mirrors the kit law: a broken table fails loud here, never silently at
        /// play time.&lt;/summary&gt;
        public static LocaleService Locale
```

```
{
    get
    {
        if (_locale == null)
        {
            TextAsset asset = Resources.Load&lt;TextAsset&gt;("cfg_i18n");
            if (asset == null) throw new ArgumentException("cfg_i18n table missing from Resources");
            I18nTable table = JsonUtility.FromJson&lt;I18nTable&gt;(asset.text);
            if (table == null || table.strings == null || table.strings.Count == 0)
                throw new ArgumentException("cfg_i18n table is empty");
            List&lt;string&gt; keys = new List&lt;string&gt;(table.strings.Count);
            List&lt;string&gt; en = new List&lt;string&gt;(table.strings.Count);
            List&lt;string&gt; zh = new List&lt;string&gt;(table.strings.Count);
            for (int i = 0; i &lt; table.strings.Count; i++)
            {
                keys.Add(table.strings[i].key);
                en.Add(table.strings[i].en);
                zh.Add(table.strings[i].zh);
            }
            _locale = new LocaleService(keys, en, zh);
        }
        return _locale;
    }
}
// wireframe palette: neutral slate board + the frozen G28 accent
// #30B4E8 (the B1294 DiffGate-frozen signal blue) as the arrow/display face
static readonly Color32 Slate = new Color32(0x24, 0x2E, 0x38, 0xFF);
static readonly Color32 SlateDark = new Color32(0x16, 0x1D, 0x24, 0xFF);
static readonly Color32 Cell = new Color32(0x3A, 0x46, 0x52, 0xFF);
static readonly Color32 Paper = new Color32(0xF2, 0xF6, 0xF9, 0xFF);
static readonly Color32 Ink = new Color32(0x10, 0x16, 0x1B, 0xFF);
static readonly Color32 Accent = new Color32(0x30, 0xB4, 0xE8, 0xFF);   // frozen accent (B1294)
static readonly Color32 Gate = new Color32(0x6E, 0xC2, 0x7A, 0xFF);
/// &lt;summary&gt;Boot wiring face: find-or-create the host in the freshly loaded
/// main scene (called by BootRunner after the scene switch; idempotent).&lt;/summary&gt;
public static PagesHost EnsureInScene()
{
    PagesHost existing = FindObjectOfType&lt;PagesHost&gt;();
    if (existing != null) return existing;
    GameObject go = new GameObject("PagesHost");
    return go.AddComponent&lt;PagesHost&gt;();
}
void Awake()
{
    BuildAll();
}
IEnumerator Start()
{
    // splash beats: hold the boot page, then hand the board the teach loop
    yield return new WaitForSeconds(0.8f);
```

```
    if (_pages.ContainsKey(PageFrozen.Names[0])) _pages[PageFrozen.Names[0]].SetActive(false);
    if (_pages.ContainsKey(PageFrozen.Names[1])) _pages[PageFrozen.Names[1]].SetActive(true);
    yield return TeachLoop();
}
// ==== build face =======================================================
public void BuildAll()
{
    AbsorbPlaceholder();
    EnsureEventSystem();
    for (int i = 0; i &lt; PageFrozen.Names.Length; i++)
    {
        string name = PageFrozen.Names[i];
        if (_pages.ContainsKey(name)) continue;
        _pages[name] = BuildPage(name);
    }
    BuildBootContent();
    BuildBoardContent();
    BuildCodexContent();
    BuildShareContent();
    _pages[PageFrozen.Names[0]].SetActive(true);   // splash first
    _pages[PageFrozen.Names[1]].SetActive(true);   // board under the splash
    _pages[PageFrozen.Names[2]].SetActive(false);  // codex overlay
    _pages[PageFrozen.Names[3]].SetActive(false);  // share overlay
}
/// &lt;summary&gt;Placeholder shell law: the ProjectSetup-generated root "Canvas"
/// (with its "S28_Main Main" label) is a skeleton face - absorb it so the
/// built pages are the only canvases (U184: zero stray canvas without scaler).&lt;/summary&gt;
void AbsorbPlaceholder()
{
    Canvas[] roots = FindObjectsOfType&lt;Canvas&gt;();
    for (int i = 0; i &lt; roots.Length; i++)
    {
        Canvas c = roots[i];
        if (c.transform.parent != null) continue;
        if (PageFrozen.IsPage(c.name)) continue;
        if (c.name == "Canvas")
        {
            if (Application.isPlaying) Destroy(c.gameObject);
            else DestroyImmediate(c.gameObject);
        }
    }
}
void EnsureEventSystem()
{
    if (FindObjectOfType&lt;EventSystem&gt;() != null) return;
    GameObject esGo = new GameObject("EventSystem");
    esGo.AddComponent&lt;EventSystem&gt;();
    esGo.AddComponent&lt;StandaloneInputModule&gt;();
}
GameObject BuildPage(string name)
```

```
{
    GameObject go = new GameObject(name);
    Canvas canvas = go.AddComponent&lt;Canvas&gt;();
    canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    // U184 per-canvas law: scaler locked to ScaleWithScreenSize at the frozen
    // portrait reference + raycaster on every page canvas.
    CanvasScaler scaler = go.AddComponent&lt;CanvasScaler&gt;();
    scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
    scaler.referenceResolution = new Vector2(PageFrozen.RefWidth, PageFrozen.RefHeight);
    scaler.matchWidthOrHeight = 0.5f;
    go.AddComponent&lt;GraphicRaycaster&gt;();
    return go;
}
// ---- shared widget helpers ----------------------------------------
static RectTransform NewRect(Transform parent, string name)
{
    GameObject go = new GameObject(name, typeof(RectTransform));
    go.transform.SetParent(parent, false);
    return go.GetComponent&lt;RectTransform&gt;();
}
static Image NewImage(Transform parent, string name, Color32 color, float w, float h, float x, float y)
{
    RectTransform rt = NewRect(parent, name);
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = new Vector2(x, y);
    Image img = rt.gameObject.AddComponent&lt;Image&gt;();
    img.color = color;
    return img;
}
static Text NewText(Transform parent, string name, string str, int size, float w, float h, float x, float y, Col
{
    RectTransform rt = NewRect(parent, name);
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = new Vector2(x, y);
    Text t = rt.gameObject.AddComponent&lt;Text&gt;();
    t.font = SharedFont;
    t.text = str;
    t.fontSize = size;
    t.alignment = TextAnchor.MiddleCenter;
    t.color = color;
    return t;
}
// cell center helper for the L1 opening board grid (7x5 at 120px pitch)
static Vector2 CellCenter(int cx, int cy)
{
    return new Vector2(-360 + cx * 120, 400 - cy * 120);
}
// ---- S99_Boot: splash with the title + slogan -----------------------
// Zero-hardcode face: every label resolves through LocaleService by key
// against the shipped cfg_i18n table.
```

```
        void BuildBootContent()
        {
            Transform page = _pages["S99_Boot"].transform;
            NewImage(page, "SplashBg", SlateDark, 0, 0, 0, 0).rectTransform.StretchFull();
            NewText(page, "PageTitleLabel", Locale.ByKey("ui_title"), 96, 900, 160, 0, 260, Paper);
            NewText(page, "PageSloganLabel", Locale.ByKey("ui_slogan"), 40, 1000, 80, 0, 100, Paper);
        }
        // ---- S01_Board: the opening board ---------------------------------
        void BuildBoardContent()
        {
            Transform page = _pages["S01_Board"].transform;
            NewImage(page, "BoardBg", Slate, 0, 0, 0, 0).rectTransform.StretchFull();
            // the L1 hook board grid (the frozen opening board the teach drive replays)
            LevelDef def = ArrowLevels.Hook();
            for (int y = 0; y &lt; def.H; y++)
                for (int x = 0; x &lt; def.W; x++)
                {
                    Vector2 c = CellCenter(x, y);
                    NewImage(page, "BoardCell_" + (y * def.W + x), Cell, 110, 110, c.x, c.y);
                }
            for (int i = 0; i &lt; def.ArrowCount; i++)
            {
                Vector2 c = CellCenter(def.Ax[i], def.Ay[i]);
                NewImage(page, "ArrowGlyph_" + i, Accent, 92, 92, c.x, c.y);
            }
            int gapIdx = 0;
            for (int y = 0; y &lt; def.H; y++)
            {
                if (def.GapRight[y])
                {
                    Vector2 c = CellCenter(def.W, y);
                    NewImage(page, "ExitGap_" + gapIdx, Gate, 60, 110, c.x, c.y);
                    gapIdx++;
                }
                if (def.GapLeft[y])
                {
                    Vector2 c = CellCenter(-1, y);
                    NewImage(page, "ExitGap_" + gapIdx, Gate, 60, 110, c.x, c.y);
                    gapIdx++;
                }
            }
            for (int x = 0; x &lt; def.W; x++)
            {
                if (def.GapTop[x])
                {
                    Vector2 c = CellCenter(x, -1);
                    NewImage(page, "ExitGap_" + gapIdx, Gate, 110, 60, c.x, c.y);
                    gapIdx++;
                }
                if (def.GapBottom[x])
```

```
            {
                Vector2 c = CellCenter(x, def.H);
                NewImage(page, "ExitGap_" + gapIdx, Gate, 110, 60, c.x, c.y);
                gapIdx++;
            }
        }
        // HUD strip: codex / share / undo entries + the level label (section-3 HUD face)
        Image undoEntry = NewImage(page, "HudUndoEntry", Paper, 300, 90, -330, -700);
        NewText(undoEntry.transform, "HudUndoLabel", Locale.ByKey("btn_undo"), 34, 280, 80, 0, 0, Ink);
        Image shareEntry = NewImage(page, "HudShareEntry", Paper, 300, 90, 0, -700);
        NewText(shareEntry.transform, "HudShareLabel", Locale.ByKey("btn_share"), 34, 280, 80, 0, 0, Ink);
        Image codexEntry = NewImage(page, "HudCodexEntry", Paper, 300, 90, 330, -700);
        NewText(codexEntry.transform, "HudCodexLabel", Locale.ByKey("btn_codex"), 34, 280, 80, 0, 0, Ink);
        NewText(page, "LevelLabel", Locale.ByKey("level_label"), 44, 400, 90, 0, 830, Paper);
        // teach caption seat: exists from the build face and stays EMPTY until
        // the teach loop fills it by CaptionKey (zero hardcoded copy at build).
        NewText(page, "TeachCaptionLabel", "", 40, 1000, 70, 0, -350, Paper);
    }
    // ---- S02_Codex: the board-type wall + the arrow-type entries --------
    void BuildCodexContent()
    {
        Transform page = _pages["S02_Codex"].transform;
        NewImage(page, "CodexBg", SlateDark, 0, 0, 0, 0).rectTransform.StretchFull();
        NewText(page, "CodexTitleLabel", Locale.ByKey("codex_title"), 64, 900, 120, 0, -780, Paper);
        for (int i = 0; i &lt; PageFrozen.CodexBoards; i++)
        {
            int col = i % 3, row = i / 3;
            NewImage(page, "CodexBoardCell_" + i, Cell, 280, 280, -320 + col * 320, 520 - row * 320);
        }
        for (int i = 0; i &lt; PageFrozen.CodexTypes; i++)
            NewImage(page, "CodexTypeEntry_" + i, Accent, 260, 120, -450 + i * 300, -480);
    }
    // ---- S03_Share: the board-clear card --------------------------------
    void BuildShareContent()
    {
        Transform page = _pages["S03_Share"].transform;
        NewImage(page, "ShareDim", Ink, 0, 0, 0, 0).rectTransform.StretchFull();
        NewImage(page, "ShareFrame", Paper, 700, 1200, 0, 0);
        Transform frame = page.Find("ShareFrame");
        NewText(frame, "ShareCodeLabel", ShareCodecCore.CodePrefix, 44, 620, 90, 0, 480, Ink);
        NewImage(frame, "ShareCardFace", Accent, 620, 620, 0, 40);
        // T014 slice2: the share card copy seats - built EMPTY (zero-hardcode
        // law), filled at runtime from the share variant pools by CaptionKey.
        NewText(frame, "ShareTitleLabel", "", 52, 560, 100, 0, 150, Paper);
        NewText(frame, "ShareBodyLabel", "", 34, 560, 80, 0, 40, Paper);
    }
    // ==== runtime faces ==================================================
    /// &lt;summary&gt;Teach pacing: walk the frozen five-beat table in wall time,
    /// show the captions on the board caption label, and run the same
    /// canonical L1 opening on the real kit sim (the sequence
```

```
/// PagesGate.RunTeachDrive replays in logical time); the save face lands
/// at the end.&lt;/summary&gt;
IEnumerator TeachLoop()
{
    _sim = new GrayboxSim();
    _ledger = new AdLedger(_sim);
    Text caption = null;
    Transform board = _pages.ContainsKey("S01_Board") ? _pages["S01_Board"].transform : null;
    if (board != null)
    {
        Transform label = board.Find("TeachCaptionLabel");
        if (label != null) caption = label.GetComponent&lt;Text&gt;();
    }
    double last = 0.0;
    int seen = 0;   // pool event scan cursor (TeachLoop is the single consumer)
    for (int i = 0; i &lt; PagesGate.TeachBeats.Length; i++)
    {
        TeachBeat beat = PagesGate.TeachBeats[i];
        double wait = beat.T - last;
        if (wait &gt; 0) yield return new WaitForSeconds((float)wait);
        last = beat.T;
        PagesGate.RunTeachBeat(_sim, i);
        if (caption != null) caption.text = Locale.ByKey(beat.CaptionKey);
        seen = ConsumePoolEvents(seen);
    }
    CaptureAndSave();
}
/// &lt;summary&gt;Pool event face (T014 slice2): consume the sim events that
/// landed since the last scan - a near_miss surfaces a pool caption on
/// the board seat, a share_card fills and shows the S03_Share card face.
/// The pick counters ARE the event counts, so every drive cycles the
/// variant pools deterministically (zero RNG).&lt;/summary&gt;
int ConsumePoolEvents(int fromIndex)
{
    if (_sim == null) return fromIndex;
    int n = _sim.Events.Count;
    for (int i = fromIndex; i &lt; n; i++)
    {
        string type = _sim.Events[i].type;
        if (type == "near_miss") ShowNearMissCaption(_nearMissPicks++);
        else if (type == "share_card") ShowShareCard(_sharePicks++, Math.Max(1, _sim.LevelsCompleted));
    }
    return n;
}
/// &lt;summary&gt;Near-miss caption pool face (T014 slice2): surface a near_miss
/// pool variant on the board caption seat; the counter is the sim's
/// near_miss event count (deterministic cycle, zero immediate repeat).&lt;/summary&gt;
public void ShowNearMissCaption(int pickCounter)
{
    Transform board = _pages.ContainsKey("S01_Board") ? _pages["S01_Board"].transform : null;
```

```
            if (board == null) return;
            Transform label = board.Find("TeachCaptionLabel");
            if (label == null) return;
            label.GetComponent&lt;Text&gt;().text = Locale.ByKey(CopyPools.Pick(CopyPools.NearMissKeys, pickCounter));
        }
        /// &lt;summary&gt;Share card pool face (T014 slice2): fill the S03_Share card
        /// title and body from the share variant pools ({N} = the board number)
        /// and show the page. The counter is the sim's share_card event count -
        /// consecutive boards cycle the variants deterministically.&lt;/summary&gt;
        public void ShowShareCard(int pickCounter, int boardNumber)
        {
            Transform share = _pages.ContainsKey("S03_Share") ? _pages["S03_Share"].transform : null;
            if (share == null) return;
            Transform frame = share.Find("ShareFrame");
            if (frame == null) return;
            Text title = frame.Find("ShareTitleLabel").GetComponent&lt;Text&gt;();
            Text body = frame.Find("ShareBodyLabel").GetComponent&lt;Text&gt;();
            string n = boardNumber.ToString();
            title.text = LocaleService.ApplySlot(Locale.ByKey(CopyPools.Pick(CopyPools.ShareTitleKeys, pickCounter)), "N
            body.text = LocaleService.ApplySlot(Locale.ByKey(CopyPools.Pick(CopyPools.ShareBodyKeys, pickCounter)), "N",
            share.gameObject.SetActive(true);
        }
        /// &lt;summary&gt;Save wiring: mirror the level progress + the dex counters
        /// into the SaveFace, then land it through ISaveService. The T008
        /// contract wired here: the AdLedger ExportStamps IS the save.adcaps
        /// face.&lt;/summary&gt;
        public void CaptureAndSave()
        {
            PagesGate.SaveFace face = new PagesGate.SaveFace();
            if (_sim != null)
            {
                face.LevelsCompleted = _sim.LevelsCompleted;
                face.LevelsSeen = _sim.LevelsSeen;
                face.ShareCards = _sim.ShareCards;
            }
            if (_ledger != null)
            {
                double[] stamps = _ledger.ExportStamps();
                for (int i = 0; i &lt; stamps.Length; i++) face.AdGrantStamps.Add((long)stamps[i]);
            }
            ServiceLocator.Get&lt;ISaveService&gt;().Save(PageFrozen.SaveSlot, PagesGate.SaveCodec.Compose(face));
        }
    }
    /// &lt;summary&gt;Full-stretch helper for backdrop rects (anchor both corners, zero offsets).&lt;/summary&gt;
    public static class RectTransformExtensions
    {
        public static RectTransform StretchFull(this RectTransform rt)
        {
            rt.anchorMin = Vector2.zero;
            rt.anchorMax = Vector2.one;
```

```
            rt.offsetMin = Vector2.zero;
            rt.offsetMax = Vector2.zero;
            return rt;
        }
    }
}
// ShareCodecCore.cs
// ShareCodecCore.cs - S1 T009 share-codec core for ArrowOut (B1831 kit-port).
// Ported under the kit-port law (G26 B1276 ShareCodecCore paradigm): every
// frozen value stays verbatim - zero new tuning.
// Frozen anchors (all traceable to the E4 kit, GrayboxSim.cs ShareCard face):
//   - code prefix "AO1-" = kit ShareCard prefix verbatim (the share-code family mark)
//   - compose face = kit ShareCard verbatim: the board-clear achievement card
//     code = one uppercase hex digit of LevelsCompleted + "-" + the board arrow
//     count - never a score form (B904 guardrail #2 analog; requirement M08
//     "AO1- prefix share code, X032 offline shape (zero server), compose/decode
//     round-trip", S05 ShareCodec ComposeCard/DecodeCard)
//   - share-form ban = kit ShareFormTokens verbatim: score / points / rank
//     never appear in any share payload face
//   - campaign face = E4 assertion 6 verbatim: the five-board campaign fires
//     exactly 4 cards (4 clears + 1 honest fail - a failed board shows off nothing)
// Structural S1 extension vs the kit: the decode face (M08 requires a true
// compose/decode round-trip; the kit only composed). The X032 offline shape is
// structural: the code alone restores the run face with zero server and zero
// external state (a fresh codec instance decodes any card identically).
// Decode is strict: null / wrong prefix / empty payload / missing separator /
// empty levels segment / non-hex levels char / empty count segment / non-digit
// count char / zero-arrow count all reject with exactly one event each.
// Zero state mutation: the codec observes the run; it never mutates anything.
// Pure C#: zero UnityEngine.
using System;
using System.Collections.Generic;
using System.Globalization;
using System.Text;
namespace Sy.ArrowOut
{
    public sealed class ShareCodecEvent
    {
        public double t;
        public string type;
        public string detail;
        public ShareCodecEvent(double t, string type, string detail) { this.t = t; this.type = type; this.detail = detai
    }
    public sealed class ShareCodecCore
    {
        // ---- frozen anchors (kit mirrors) ----
        public const string CodePrefix = "AO1-";      // kit ShareCard prefix verbatim
        public static readonly string[] ShareFormTokens = { "score", "points", "rank" };   // kit verbatim
        public double SimTime;
        public readonly List&lt;ShareCodecEvent&gt; Events = new List&lt;ShareCodecEvent&gt;();
```

```
public int CardsComposed, CardsDecoded, CardsRejected;
public void Log(string type, string detail) { Events.Add(new ShareCodecEvent(SimTime, type, detail)); }
public int CountEvents(string type)
{
    int n = 0;
    for (int i = 0; i &lt; Events.Count; i++) if (Events[i].type == type) n++;
    return n;
}
// ==== share-form scan face (kit ContainsAny over the frozen mirror) ====
public static bool ContainsShareFormToken(string text)
{
    if (text == null) return false;
    for (int i = 0; i &lt; ShareFormTokens.Length; i++) if (text.Contains(ShareFormTokens[i])) return true;
    return false;
}
// ==== compose from the run face: the pure restore form (no gate, no
// events) - the X032 offline half and the round-trip pair of DecodeCard.
public static string ComposeOwned(int levelsCompleted, int arrowCount)
{
    if (levelsCompleted &lt; 0 || arrowCount &lt; 0) return null;
    StringBuilder sb = new StringBuilder(CodePrefix);
    sb.Append(levelsCompleted.ToString("X", CultureInfo.InvariantCulture));
    sb.Append('-').Append(arrowCount);
    return sb.ToString();
}
// ==== compose the board-clear card = kit ShareCard face verbatim: every
// board clear shows the escape off. The card is a board-clear achievement
// card; the share-form words (score/points/rank) stay out of every face.
public string ComposeCard(GrayboxSim sim)
{
    if (sim == null)
    {
        CardsRejected++;
        Log("share_rejected", "no run to show off");
        return null;
    }
    string code = ComposeOwned(sim.LevelsCompleted, sim.Arrows.Length);
    CardsComposed++;
    Log("share_card", "board clear achievement card code=" + code + " (all arrows out)");
    return code;
}
// ==== decode = structural extension (M08 round-trip face): strict parse,
// zero server, zero external state. Returns { levelsCompleted, arrowCount }
// or null. Rejects null / wrong prefix / empty payload / missing
// separator / empty levels / non-hex levels char / empty count /
// non-digit count char / zero-arrow count - one share_decode_rejected
// event per reject.
public int[] DecodeCard(string code)
{
    if (code == null) return RejectDecode("null code");
```

```
            if (!code.StartsWith(CodePrefix, StringComparison.Ordinal))
                return RejectDecode("wrong or missing AO1- prefix: '" + code + "'");
            string payload = code.Substring(CodePrefix.Length);
            if (payload.Length == 0) return RejectDecode("empty payload after the AO1- prefix");
            int sep = payload.IndexOf('-');
            if (sep &lt; 0) return RejectDecode("missing board-size separator in '" + payload + "'");
            string levelsSeg = payload.Substring(0, sep);
            string countSeg = payload.Substring(sep + 1);
            if (levelsSeg.Length == 0) return RejectDecode("empty levels segment before the separator");
            if (countSeg.Length == 0) return RejectDecode("empty arrow-count segment after the separator");
            int levels = 0;
            for (int i = 0; i &lt; levelsSeg.Length; i++)
            {
                char ch = levelsSeg[i];
                int v;
                if (ch &gt;= '0' &amp;&amp; ch &lt;= '9') v = ch - '0';
                else if (ch &gt;= 'A' &amp;&amp; ch &lt;= 'F') v = ch - 'A' + 10;
                else return RejectDecode("non-hex char '" + ch + "' in the levels segment at slot " + i);
                levels = levels * 16 + v;
            }
            int count = 0;
            for (int i = 0; i &lt; countSeg.Length; i++)
            {
                char ch = countSeg[i];
                if (ch &lt; '0' || ch &gt; '9')
                    return RejectDecode("non-digit char '" + ch + "' in the arrow-count segment at slot " + i);
                count = count * 10 + (ch - '0');
            }
            if (count == 0) return RejectDecode("arrow count must be at least 1");
            CardsDecoded++;
            Log("share_decode", "card decoded: levels=" + levels + " arrows=" + count
                + " (offline shape, zero server)");
            return new int[] { levels, count };
        }
        int[] RejectDecode(string reason)
        {
            CardsRejected++;
            Log("share_decode_rejected", reason);
            return null;
        }
    }
}
```